

Садржај

Списак слика	2
Списак табела	2
1 Увод	3
2 Визуелна одометрија	4
2.1 <i>SVO</i>	6
2.2 <i>ORB-SLAM3</i>	8
2.3 <i>TSformer-VO</i>	10
2.4 Окружење	12
3 Прикупљање података	13
3.1 Робот	13
3.1.1 Диференцијални погон	14
3.1.2 Одометријски систем	14
3.2 <i>Rosbag</i>	16
3.3 Камера	17
3.3.1 Калибрација камере	17
3.4 Секвенце	19
3.4.1 Процена референтних путања	21
4 Резултати и дискусија	23
4.1 Евалуација одометрије	23
4.1.1 Поравнавање процењене и референтне путање	24
4.1.2 <i>Utneyata</i> метода	24
4.2 Резултати <i>SVO</i>	25
4.3 Резултати <i>ORB-SLAM3</i>	26
4.4 Резултати <i>TSformer-VO</i>	27
4.5 Поређење алгоритама	28
5 Закључак	30
Литература	33
Прилог	33

Списак слика

2.1	Структура алгоритма <i>SVO</i> : процес за процену кретања и процес за мапирање (преузето из [6])	7
2.2	Три корака процене кретања у <i>SVO</i> алгоритму (преузето из [6])	7
2.3	Компоненте система <i>ORB-SLAM3</i> (преузето из [15])	9
2.4	Структура модела <i>TSformer-VO</i> (преузето из [18])	10
2.5	Визуелизација поновљених кретања за случај $N_f = 3$ (преузето из [18])	11
2.6	Блок <i>Transformer</i> енкодера са подељеном просторно-временском самопа- жњом (преузето из [18])	11
3.1	Робот тима +381 <i>Robotics</i> који је коришћен за прикупљање скупа података . .	13
3.2	Кинематски модел диференцијалног робота	16
3.3	Слика <i>ChArUco</i> табле која је коришћена за калибрацију	18
3.4	Изглед лабораторије забележен камером на роботу током снимања секвенце .	19
3.5	Изглед стола за такмичење Еуробот 2026	19
3.6	Референтна путања робота у праволинијским секвенцама	20
3.7	Референтна путања робота у криволинијским секвенцама	21
3.8	Секвенца добијања референтних тачака	22
4.1	Путање процењене алгоритмом <i>SVO</i>	25
4.2	Путање процењене алгоритмом <i>ORB-SLAM3</i>	26
4.3	Путање процењене алгоритмом <i>TSformer-VO</i>	27
4.4	Процењене путање свих анализираних алгоритама	28

Списак табела

4.1	Резултати евалуације визуелне одометрије	29
-----	--	----

1 Увод

Праћење сопственог положаја у времену је један од најважнијих проблема у мобилној роботизици. Недовољно брза и стабилна процена негативно утиче на управљање кретањем робота, док се константне грешке нагомилавају и полако се губи прецизна позиција робота. Одометрија је основни приступ процени кретања и њен задатак је да процени промену положаја у времену на основу мерења неког сензора. Локализација има за циљ да одреди положај у познатом окружењу, чиме компензује нагомилану грешку одометрије. Што је грешка одометрије мања, то је задатак локализације једноставнији.

Сензори на основу чијих мерења се процењује промена положаја могу да се поделе на проприоцептивне и екстероцептивне. Проприоцептивни сензори мере унутрашња стања робота, као што су: положај, брзина и убрзање у зглобовима, а њихова мерења могу без много обраде да се користе за одометрију. Екстероцептивни сензори сакупљају податке о спољашњем окружењу, као што су: слике окружења и удаљеност до препрека, које могу да се користе и за локализацију.

У овом раду су истражене методе процене кретања на основу секвенце слика које прикупља камера. Такав приступ се назива визуелна одометрија. Монокуларна визуелна одометрија користи слике са једне камере. Због недостатка информације о дубини и неодређености апсолутне скале, ово представља веома ограничен приступ одометрији.

С обзиром на то да је главни улаз у ВО секвенца слика спољашњег окружења, веома је важно да се рад алгоритама ВО анализира у одговарајућем окружењу. Због тога је направљен експериментални скуп података на мобилном роботу. Он подразумева мобилног робота који се креће у лабораторијском окружењу и секвенци слика које су прикупљене камером која је постављена на роботу.

На експерименталном скупу података су анализирана три алгорита за монокуларну визуелну одометрију: *SVO*, *ORB-SLAM3* и *TSformer-VO*. Процењене путање су упоређене са референтном путањом, која је настала обрадом одометрије са пасивних точкова.

2 Визуелна одометрија

Визуелна одометрија (ВО) представља процес инкременталне процене кретања камере. Термин је увео Нистер 2004. године због сличности са одометријом са тачкова [1, 2]. Одометрија са тачкова процењује кретање возила интеграцијом прираштаја углова тачкова. Визуелна одометрија инкрементално процењује кретање камере на основу промене у сликама које тај покрет индукује [2].

Проблем реконструкције релативних положаја камере и тродимензионалне структуре сцене на основу скупа слика познат је као *SfM* (*Structure from Motion*). Проблем реконструкције се може решавати на основу секвенцијално уређених или неуређених скупова слика. *Bundle adjustment* (BA) је процес оптимизације реконструисане структуре и положаја камере. Рачунски је захтеван и време извршавања расте са бројем слика. У општем случају, BA се може извршавати накнадно. Визуелна одометрија представља посебан случај *SfM* проблема. Фокусира се на секвенцијалној процени кретања камере у реалном времену. Како нова слика пристиже, тако се рачуна и нова поза. *Bundle adjustment* у визуелној одометрији може да се користи за оптимизацију локалне процене путање. Како би рад алгоритма остао у реалном времену, мора се водити рачуна о времену извршавања оптимизације [1].

Успешност рада алгоритама визуелне одометрије директно зависи од садржаја слика спољашњег окружења. За процену кретања најважнији су следећи фактори:

- осветљеност сцене,
- статична сцена,
- текстура и
- довољно преклапање узастопних слика.

Осветљеност сцене и текстура омогућавају поуздано издвајање и праћење визуелних карактеристика. У потпуно статичној сцени, свака промена у слици настаје искључиво као последица кретања камере. Објекти у сцени који се померају представљају сметњу при процени кретања. Довољно преклапање сцене омогућава да се исте визуелне карактеристике пронађу у узастопним сликама. На основу промене положаја тих визуелних карактеристика врши се процена кретања.

Процена дубине сцене представља основу алгоритама визуелне одометрије. У случају стерео ВО, дубина сцене се рачуна на основу пиксела који се подударају у сликама леве и десне камере. Положај тачке у сцени се рачуна на основу:

- разлике положаја пиксела који одговара тој тачки у обе слике (диспаритета),
- познатог растојања између леве и десне камере (базне дистанце) и
- унутрашњих параметара камера

У изразу (1) је представљен прорачун положаја тачке у сцени на основу стерео камере [3]. Са x , y и z је означен положај тачке у координатном систему камере, x_c и y_c представљају координате пиксела који одговарају тачки, b је базна дистанца, d је диспаратет, а f је жижна

даљина изражена у пикселима.

$$\begin{bmatrix} x & y & z \end{bmatrix} = \begin{bmatrix} \frac{x_c \cdot z}{f} & \frac{y_c \cdot z}{f} & \frac{b \cdot f}{d} \end{bmatrix} \quad (1)$$

Грешка процене дубине расте са квадратом удаљености између тачке и камере [4]. Због тога се у случају великих удаљености у сцени (много веће од базне дистанце) стерео визуелна одометрија своди на случај монокуларне [2]. Монокуларна визуелна одометрија не може директно да измери дубину тачака у сцени. Уместо стерео пара, користи се временски пар слика. Тачка у сцени се посматра из два различита положаја камере. Њена дубина се реконструише триангулацијом на основу кореспонденције у узастопним сликама. С обзиром на то да не постоји податак као базна дистанца на основу којег би се одредила апсолутна скала, алгоритми за монокуларну визуелну одометрију могу да процене кретање само до фактора скале [2].

Методе визуелне одометрије могу првенствено да се поделе на геометријске и методе засноване на дубоком учењу. Геометријске методе могу даље да се поделе на методе засноване на карактеристичним тачкама и директне [5]. Директне методе процењују структуру и кретање на основу интензитета пиксела у слици. Процена се заснива на минимизацији фотометријске грешке [6]. Фотометријска грешка представља разлику интензитета између одговарајућих пиксела у пару слика. Фотометријска грешка δI је представљена у изразима (2) - (4):

$$\mathbf{p}_u = \pi^{-1}(\mathbf{u}, d_u) \quad (2)$$

$$\mathbf{u}' = \pi(\mathbf{T} \cdot \mathbf{p}_u) \quad (3)$$

$$\delta I(\mathbf{T}, \mathbf{u}) = I_k(\mathbf{u}') - I_{k-1}(\mathbf{u}) \quad (4)$$

Координате пиксела у претходној слици су представљене са $\mathbf{u}$. Са $\mathbf{p}_u$ је означен положај 3D тачке која одговара том пикселу. Дубина тог пиксела је d_u . Са π^{-1} је представљена инверзна пројекција, која на основу координата и дубине пиксела одређује положај 3D тачке. $\mathbf{u}'$ представља координате пиксела у тренутној слици. Функција пројекције која 3D тачку претвара у координате у слици је π . Релативна трансформација између претходног и тренутног положаја камере је $\mathbf{T}$. Интензитети пиксела у тренутној и претходној слици су означени са I_k и I_{k-1} .

Методе засноване на карактеристичним тачкама експлицитно детектују и упарују карактеристике. Карактеристика је образац у слици који се разликује од свог непосредног окружења у погледу интензитета, боје и текстуре. Регион око детектованих карактеристика се репрезентује дескрипторима, на основу којих се карактеристике упарују [2]. У овим методама, кретање се процењује минимизацијом репројекционе грешке [7]. Репројекциона грешка представља разлику између измерених координата карактеристичне тачке у слици и пројектованих координата одговарајуће 3D тачке. Пројектоване координате представљају положај карактеристичне тачке у слици према процењеном кретању. Репројекциона грешка је представљена у изразу (5).

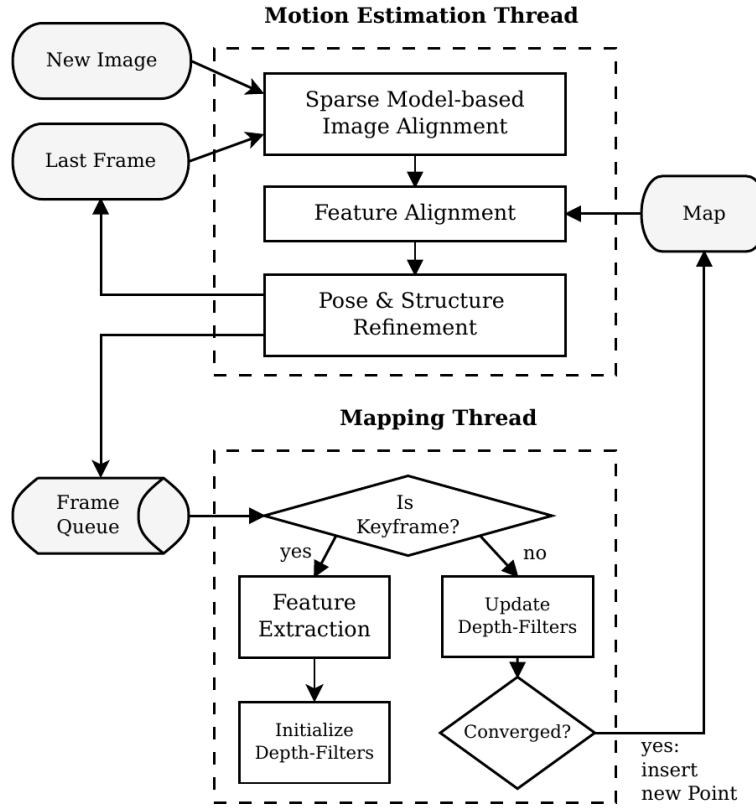
$$\mathbf{e} = \mathbf{u} - \mathbf{u}' \quad (5)$$

Визуелна одометрија и визуелна симултана локализација и мапирање (*vSLAM*) су често у литератури представљени као синоними [8]. Оба представљају потенцијално решење за проблем процене положаја. Међутим, потребно их је разликовати. Визуелна одометрија је фокусирана на процену локалне путање камере у реалном времену. Фокус *vSLAM* алгоритама је на глобалној процени путање, односно на изградњи глобално конзистентне мапе и локализацији у њој. Важан део *vSLAM* алгоритама је детекција и затварање петљи, који омогућава компензацију нагомилане грешке (дрифта) [8]. Основни део *vSLAM* система представља алгоритам за визуелну одометрију [8].

2.1 SVO

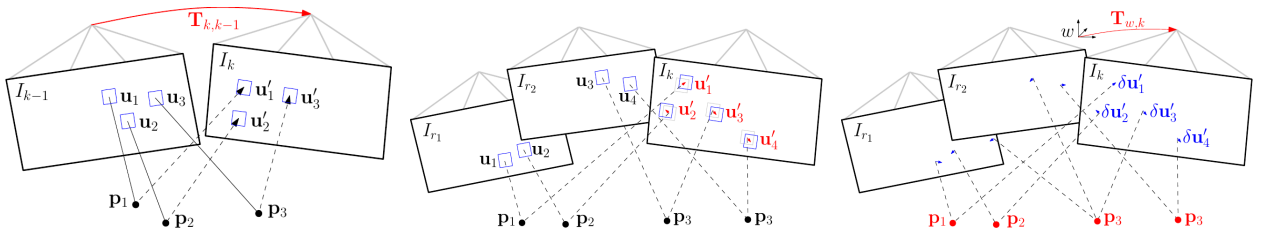
SVO (*Semi-Direct Visual Odometry*) је алгоритам за монокуларну визуелну одометрију који је развијен у радовима [6, 9]. Првенствено је развијен за коришћење у микро летелицама (*Micro Aerial Vehicles, MAVs*), где су рачунски ресурси веома ограничени и где је потребна велика фреквенција процене кретања. Алгоритам користи кореспонденцију карактеристика која је имплицитан резултат директне процене кретања. С обзиром на то да кореспонденција није резултат експлицитног издвајања и упаривања карактеристика [6], у овом раду је *SVO* издвојен као представник директних метода визуелне одометрије. Издвајање карактеристика се врши само када се одабере нова кључна слика, ради иницијализације нових 3D тачака. Предност таквог приступа је прескакање рачунски захтевног процеса издвајања карактеристика у сваком фрејму. Чим се заврши кореспонденција карактеристика и процена иницијалне позе камере, алгоритам наставља са коришћењем само претходно иницијализованих 3D тачака. За разлику од ранијих директних метода, које су пратиле мали број великих равних површи [10, 11, 12, 13], *SVO* прати стотине малих подручја слике (*patch*) величине 4×4 пиксела [9].

Структура алгоритма је приказана на слици 2.1. Алгоритам је подељен у два паралелна процеса. Процес за процену кретања обрађује сваку нову слику и процењује позу у односу на мапу. Процес за мапирање проширује мапу новим 3D тачкама и није везан за рад у реалном времену [6].



Слика 2.1 Структура алгоритма *SVO*: процес за процену кретања и процес за мапирање (преузето из [6])

Процена кретања се састоји из три корака, који су приказани на слици 2.2. Први корак представља иницијализацију позе. Релативна поза камере у односу на претходну слику се процењује минимизацијом фотометријске грешке између подручја слике који одговарају истим 3D тачкама. У другом кораку се фотометријска грешка додатно минимизује. Врши се оптимизација положаја сваког издвојеног подручја појединачно. Овим кораком се нарушавају епиполарна ограничења, како би се добила већа корелација између подручја у слици. Последњим кораком се оптимизују поза камере и положаји 3Д тачака минимизацијом репројекционе грешке.



Слика 2.2 Три корака процене кретања у *SVO* алгоритму (преузето из [6])

У овом раду је коришћена отворена имплементација *SVO Pro* [14], која представља проширену верзију изворног алгоритма [6, 9]. У односу на изворни алгоритам, проширена верзија доноси следећа проширења:

- подршку за различите типове камера, укључујући перспективне и камере са широкоугаоном пројекцијом,
- подршку за монокуларне и стерео поставке,

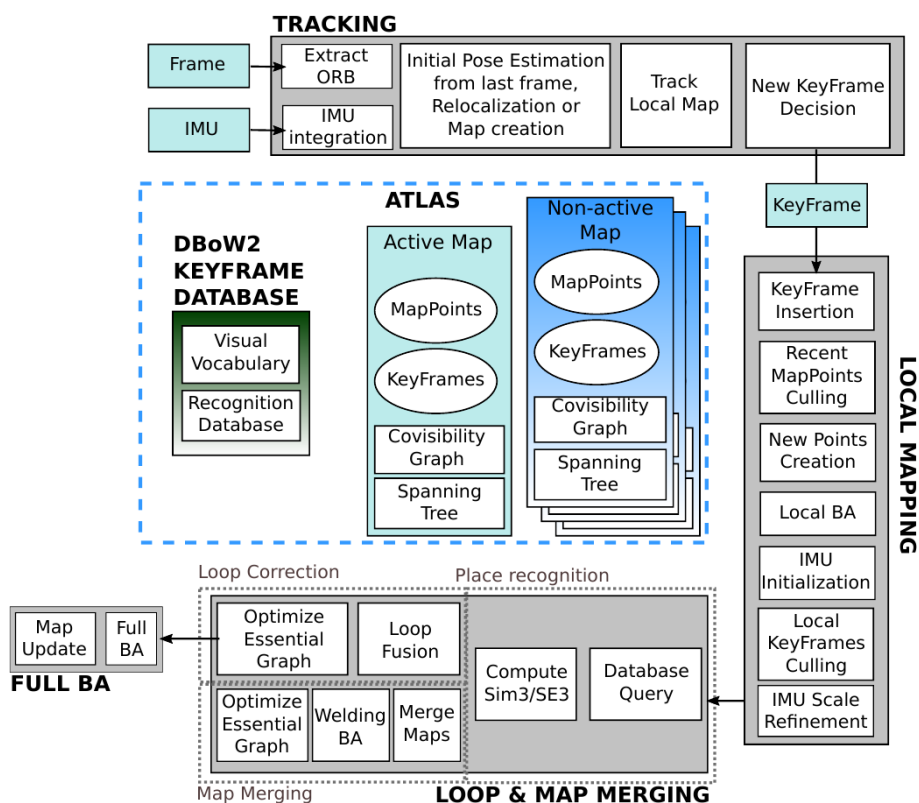
- праћење ивичних сегмената (*edgelets*),
- укључивање претходне процене кретања у поравнавање слике,
- оптимизацију засновану на клизном прозору,
- глобалну оптимизацију мапе и
- затварање петљи.

У овом раду је коришћен искључиво монокуларни визуелни део алгоритма, без наведених проширења.

2.2 *ORB-SLAM3*

ORB-SLAM3 је систем за *vSLAM* који обухвата чисто визуелни, визуелно-инерцијални и *SLAM* са више мапа. Подржава монокуларне, стерео и *RGB-D* камере [15]. Наследник је *ORB-SLAM* [16] и *ORB-SLAM2* [17], од којих преузима основну архитектуру. Аутори разликују три нивоа информација који су значајни за праћење кретања. Краткорочне информације се користе за упаривање елемената мапе из претходних неколико секунди. Елементи окружења се прате док су у погледу и заборављају се чим нестану из погледа. Ово је једини ниво информација који се користи у већини алгоритама визуелне одометрије [15]. Средњерочне информације се користе за упаривање тренутне слике са елементима из окружења који се налазе близу камере и чија је нагомилана грешка мала. Овима се дрифт своди на нулу док се камера креће кроз претходно мапирано подручје [15]. Дугорочне информације су засноване на препознавању окружења. Тренутна слика се упарује са претходно посећеним окружењем, без обзира на нагомилану грешку. Односно, врши се детекција и затварање петље. Овај ниво асоцијације података омогућава спајање неповезаних мапа и релокализацију у случају губитка праћења [15]. *ORB-SLAM3* је изабран не само зато што је представник метода заснованих на карактеристичним тачкама, већ и како би се обрадио утицај локализације на успешност визуелне одометрије.

ORB-SLAM3 се састоји из три паралелна процеса: процес за праћење, процес за локално мапирање и процес за спајање мапа. Поред њих, битан део алгоритма је структура *Atlas*. Компоненте алгоритма су приказане на слици 2.3 и описане у наставку.



Слика 2.3 Компоненте система *ORB-SLAM3* (преузето из [15])

Процес за праћење обрађује податке са сензора и процењује позу у односу на активну мапу. Поза се процењује минимизацијом репројекционе грешке упарених карактеристика у реалном времену. Такође, процес за праћење одређује да ли тренутна слика постаје кључна. У случају губитка праћења, врши се релокализација у свим мапама из *Atlas*-а. Уколико је релокализација успешна, праћење се наставља и активна мапа се мења уколико је то потребно. У супротном, након одређеног времена, активна мапа постаје неактивна и нова мапа се иницијализује као активна [15].

Процес за локално мапирање додаје нове и отклања редундантне кључне слике и 3D тачке из активне мапе. Поред тога врши оптимизацију мапе локалним *bundle adjustment*-ом [15].

Процес за спајање мапа детектује заједничке регионе између активне мапе и целог *Atlas*-а. Уколико заједнички регион припада активној мапи, врши се корекција петље. Уколико припада некој другој мапи, врши се спајање мапа. Глобални *bundle adjustment* се покреће у засебном процесу након корекције петље [15].

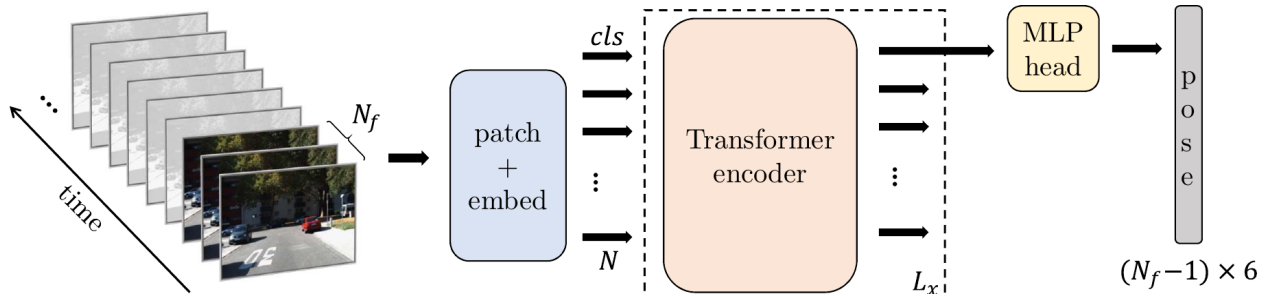
Atlas представља структуру за репрезентацију више неповезаних мапа. Једна мапа је активна у сваком тренутку. Процес за праћење локализује камеру на основу пристиглих слика у активној мапи. Локална мапа се континуално оптимизује и проширује у процесу за локално мапирање [15].

У овом раду је коришћен *ORB-SLAM3* у монокуларном режиму, са омогућеним затварањем петљи и спајањем мапа.

2.3 TSformer-VO

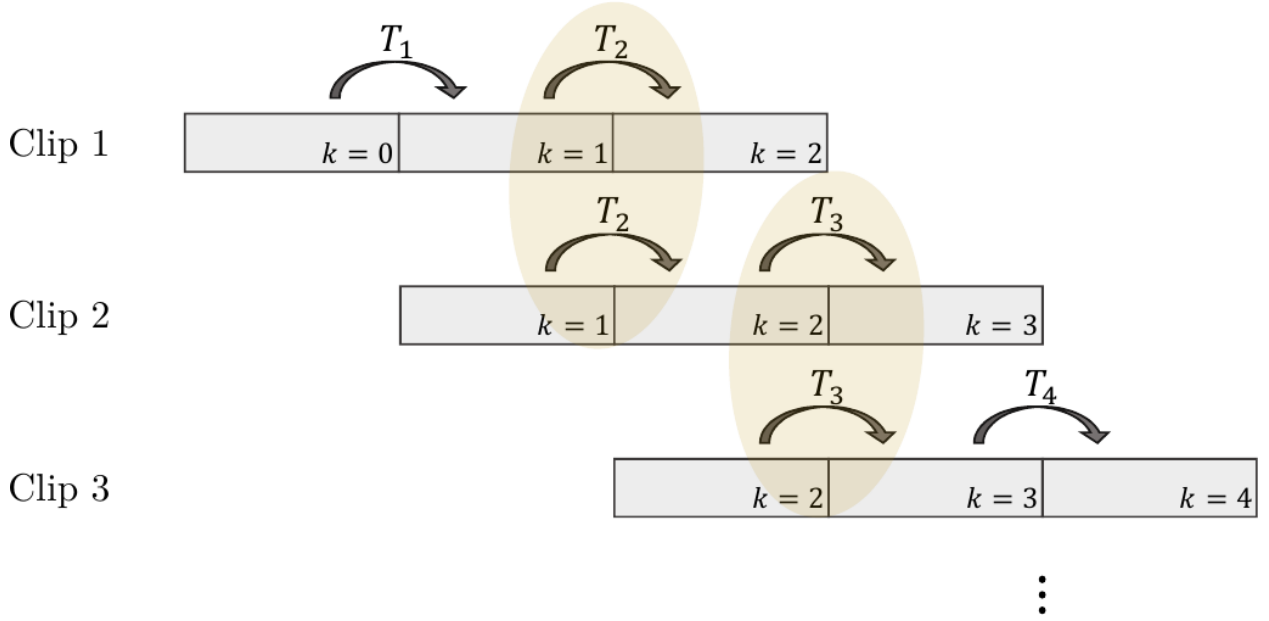
TSformer-VO је метода монокуларне визуелне одометрије заснована на дубоком учењу, која проблем процене кретања посматра као задатак разумевања видеа [18]. Модел регресијом процењује релативне позе камере на основу кратког скупа узастопних слика. Модел је заснован на *TimeSformer* архитектури за препознавање акција. Користи механизам самопажње за издвајање просторно-временских карактеристика, које се користе за процену поза камере. За разлику од геометријских метода, модел може имплицитно да научи скалу кретања из скупа података за обучавање.

Структура модела је приказана на слици 2.4. Скуп од N_f узастопних слика представља улазни податак: $\mathbf{X} \in \mathbb{R}^{N_f \times C \times H \times W}$, где су C , H и W број канала, висина и ширина слике. Свака слика се дели на $N = HW/P^2$ непреклапајућих делова величине $P \times P$ пиксела. Они се затим развијају у векторе и линеарним пресликавањем уграђују у токене димензије E_d . Поступак поделе слике на делове и њиховог уграђивања у токене представљају стандардни приступ који је уведен у архитектури *Vision Transformer* [19]. Сваком токenu се додаје информација о његовој позицији, представљена научивим позиционим вектором. На почетак низа се, по узору на *BERT* [20], додаје и посебан научиви токен: класни токен. Он се користи за прикупљање информација из свих делова улазних слика, како би се избегао бијас при избору излаза једног конкретног дела слике. Низ токена пролази кроз L_x блокова *Transformer* енкодера, а класни токен се на крају прослеђује излазном вишеслојном перцептрону (*MLP head*). С обзиром на то да су за једну релативну позу потребне две узастопне слике, за исечак од N_f слика излаз чини $N_f - 1$ поза. Односно, излазни слој има димензију $6(N_f - 1)$.



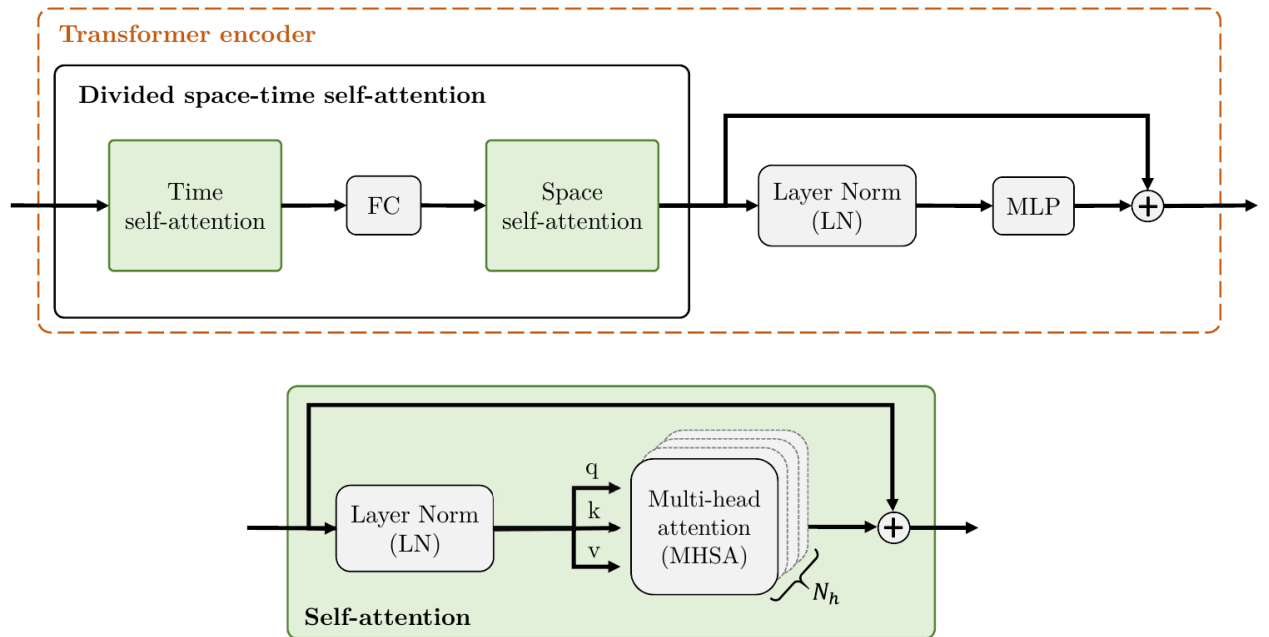
Слика 2.4 Структура модела *TSformer-VO* (преузето из [18])

Улазни скупови слика се преклапају за $N_f - 1$ слика. Због тога иста кретања могу да се процене више пута. Клипови се формирају као кружни бафер величине N_f . Свако кретање представља усредњену вредност процена из различитих клипова. Овај поступак уводи мало кашњење, које је занемарљиво због малог броја слика у клипу. На слици 4 је приказана визуелизација улазних клипова и поновљених кретања за случај $N_f = 3$.



Слика 2.5 Визуелизација поновљених кретања за случај $N_f = 3$ (преузето из [18])

У *TimeSformer* архитектури су дефинисане две архитектуре самопажње за издвајање просторно-временских карактеристика. У комбинованој просторно-временској самопажњи (*joint space-time self-attention*), сви токени из клипа се прослеђују моделу. На овај начин се истовремено разматрају сви парови интеракција између токена у простору и времену. Разматрање других интеракција кроз цео видео може довести до велике рачунске сложености. Због рачунских ограничења у примени визуелне одометрије у реалном времену, аутори су изабрали другу архитектуру самопажње. Подељена просторно-временска самопажња (*divided space-time self-attention*) подразумева раздвајање временске и просторне обраде. Прво се разматрају токени са истим просторним индексом дуж временске осе, а затим токени из исте слике дуж просторне осе [18, 21]. Блок енкодера је приказан на слици 2.6.



Слика 2.6 Блок *Transformer* енкодера са подељеном просторно-временском самопажњом (преузето из [18])

TSformer-VO модели су обучени на *KITTI* скупу података [18, 22]. У овом раду су коришћени модели из оригиналне имплементације, без додатног обучавања на сопственом скупу података.

2.4 Окружење

С обзиром на то да су у овом раду коришћени алати и алгоритми који су развијени независно једни од других, јавила се потреба за коришћењем изолованих развојних окружења. Уколико би се све инсталирало у једном окружењу, појавили би се конфликти између верзија пакета, *Python* интерпретера и библиотека. Коришћени су различити нивои изолације, у зависности од захтева појединачних алата. У наставку су наведена три алата која су коришћена за изолацију окружења:

- *Podman* контејнери,
- *Conda* и
- *Python venv*.

Контејнери представљају потпуну изолацију са комплетним оперативним системом и системским библиотекама. Окружење изоловано у контејнеру независно је од *host* оперативног система. *Conda* представља средњи ниво изолације и омогућава потпуну изолацију *Python* окружења од системског. *Python venv* користи системски *Python* интерпретер и омогућава само одвајање инсталираних *Python* пакета. Избор алата који се користи за изолацију датог окружења зависи од системских зависности. Уколико је потребан одређен оперативни систем, користи се контејнер. Уколико је потребна другачија верзија *Python* интерпретера са свим осталим пакетима, користи се *Conda*. Уколико су потребне одређене верзије појединачних *Python* пакета, без потребе за променом интерпретера, користи се *Python venv*.

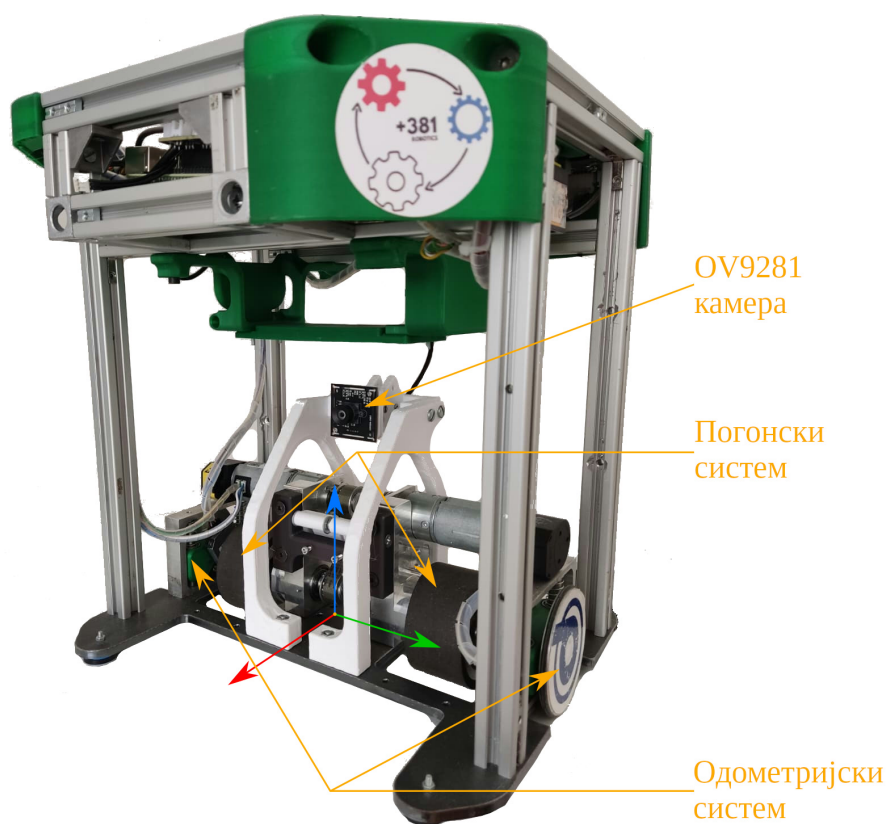
Целокупно радно окружење је подељено у два контејнера, у којима су даље изолована засебна *Conda* и *Python venv* окружења. Први је коришћен за *SVO* и *ORB-SLAM3* алгоритме, који захтевају *ROS 1 Noetic* дистрибуцију роботског оперативног система. С обзиром на то да су *ROS* дистрибуције директно везане за верзије *Ubuntu* оперативног система, у првом контејнеру је коришћен *Ubuntu 20.04*. Поред тога, први контејнер је коришћен и за обраду *rosbag* датотека. Други контејнер је коришћен за *TSformer-VO* алгоритам, евалуацију резултата и анализу и обраду података. Иако ниједна од претходно наведених намена није захтевала употребу специфичног оперативног система и није било неопходно користити још један контејнер, коришћен је модернији *Ubuntu 24.04* због новијих верзија *Python*-а и библиотека.

3 Прикупљање података

Како би се испитала примена ВО у мобилној роботизи, прикупљен је експериментални скуп података. Експерименталну поставку чине робот са диференцијалним погоном и круто постављеном камером, док се креће кроз различите секвенце кретања у дефинисаном простору.

3.1 Робот

За прикупљање података коришћен је мобилни робот развијен у оквиру тима *+381 Robotics* за такмичење Еуробот Србија 2026. Робот није коришћен у целокупном саставу као на такмичењу, већ су остављени само релевантни системи. Поред тога, на робота је постављена камера која је коришћена за снимање података над којима су тестирани алгоритми визуелне одометрије. Камера је постављена директно изнад локалног координатног система базе робота и оријентисана је у правцу кретања. С обзиром на то да се робот креће у равни, оваква поставка омогућава да се прескочи корак рачунања трансформације камера-робот. Односно, кретање које је процењено као резултат алгоритама визуелне одометрије може директно да се пореди са референтном путањом. Робот који је коришћен у експерименту је приказан на слици 3.1. На слици је означен локални координатни систем и камера, као и системи релевантни за прикупљање података који су описани у наставку.



Слика 3.1 Робот тима *+381 Robotics* који је коришћен за прикупљање скупа података

3.1.1 Диференцијални погон

Робот користи диференцијални погонски систем. Овакав тип погона подразумева два независно актуисана точка који су међусобно саосни. По ободу точкова се налази мека гума, чиме се обезбеђује кретање уз минимално бочно проклизавање. Кретање робота је ограничено нехолономним ограничењем, према којем је бочна брзина базе увек једнака нули.

У зависности од брзина точкова, овакав погонски систем омогућава три основна типа кретања:

- праволинијска транслација,
- ротација око центра робота и
- криволинијско кретање.

Уколико су линеарне брзине точкова у тачкама контакта са подлогом једнаке по интензитету и смеру, робот врши праволинијску транслацију. Уколико су брзине точкова истог интензитета, а супротног смера, робот ротира око центра, односно око z -осе локалног координатног система базе. Различите брзине точкова доводе до криволинијског кретања базе робота.

3.1.2 Одометријски систем

Уколико би се брзине погонских точкова користиле за одометрију, конструкција робота би била значајно једноставнија. Међутим, погонски точкови су подложни проклизавању услед преношења силе на подлогу, што би довело до значајне грешке при процени положаја. Због тога је одометријски систем потпуно одвојен од погонског. Састоји се од два инкрементална енкодера који се налазе на пасивним точковима. На пасивне точкове делује сила опруге, чиме се обезбеђује стални контакт са подлогом.

Позиција базе робота се израчунава на основу кинематског модела који је приказан у једначинама (6) - (12).

$$v_r = \omega_r \cdot \frac{d}{2} \quad (6)$$

$$v_l = \omega_l \cdot \frac{d}{2} \quad (7)$$

$$v = \frac{v_r + v_l}{2} \quad (8)$$

$$\omega = \frac{v_r - v_l}{D} \quad (9)$$

$$x_c^{k+1} = x_c^k + v \cdot \cos(\varphi_c^k + \frac{\omega \cdot \Delta t}{2}) \cdot \Delta t \quad (10)$$

$$y_c^{k+1} = y_c^k + v \cdot \sin(\varphi_c^k + \frac{\omega \cdot \Delta t}{2}) \cdot \Delta t \quad (11)$$

$$\varphi_c^{k+1} = \varphi_c^k + \omega \cdot \Delta t \quad (12)$$

Угаоне брзине пасивних точкова се добијају на основу мерења енкодера у константним временским одбирцима. Мери се прираст броја импулса, односно промена угла пасивних точкова, у интервалу Δt . Тиме добијамо просечну брзину пасивних точкова за тај интервал. При праволинијском кретању, оријентација робота се не мења и просечна брзина

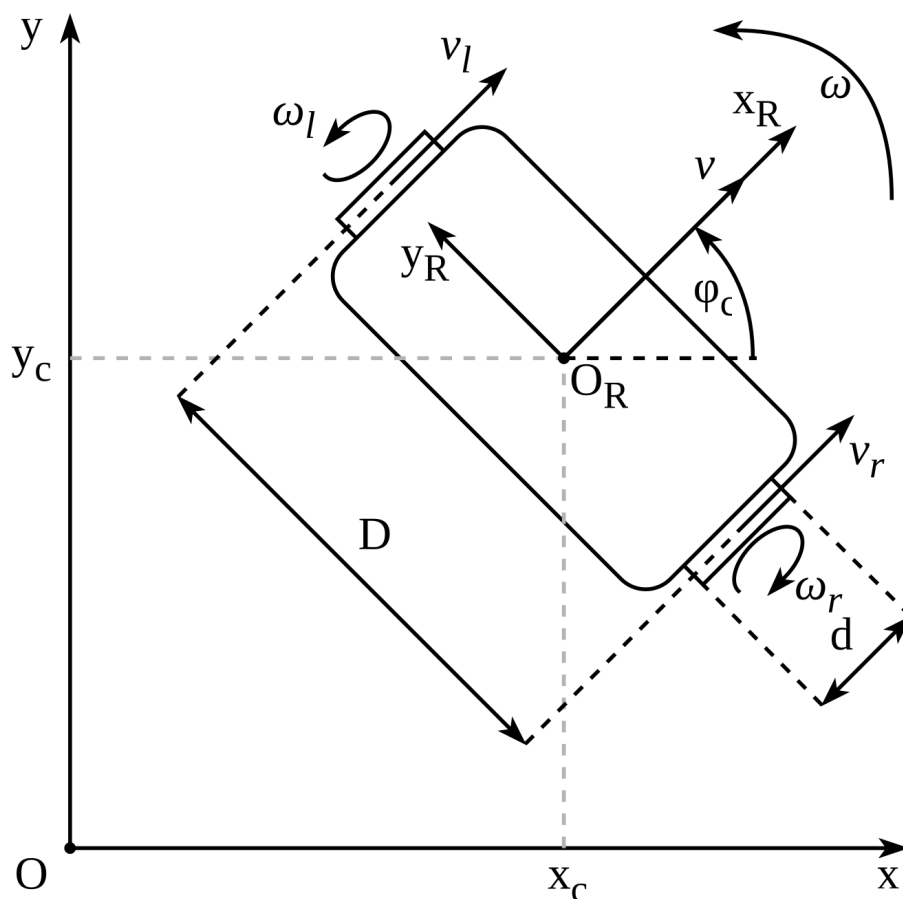
добијена из енкодера може тачно да представља кретање у том интервалу. При ротацији око центра, робот не мења позицију већ само оријентацију, па коришћење просечне брзине не утиче на процену позиције. Код криволинијског кретања се истовремено мења и оријентација и позиција. Током интервала Δt робот прелази пут при различитим оријентацијама, а у моделу се рачуна као да је прешао пут по просечној оријентацији. То представља извор грешке који утиче на процену положаја. Како би се ограничио утицај грешке криволинијског кретања, коришћен је веома кратак временски интервал $\Delta t = 1ms$. Угаоне брзине пасивних точкова се рачунају према једначини (13).

$$\omega_{r,l} = \frac{\Delta n_{r,l}}{\Delta t \cdot N} \cdot 2\pi \quad (13)$$

Ознаке коришћене у кинематском моделу имају следеће значење:

- O, x, y - референтни координатни систем
- O_R, x_R, y_R - локални координатни систем робота
- x_c, y_c, φ_c - положај и оријентација робота у референтном систему
- v - линеарна брзина базе робота
- ω - угаона брзина базе робота
- ω_r, ω_l - угаона брзина десног и левог одометријског точка
- v_r, v_l - линеарна брзина десног и левог одометријског точка
- D - растојање између одометријских точкова
- d - пречник одометријских точкова
- Δt - временски интервал између два одбирка
- $k, k + 1$ - тренутни и наредни одбирак
- $\Delta n_{r,l}$ - број импулса десног и левог енкодера у временском интервалу Δt
- N - број импулса енкодера за један пун круг

На слици 3.2 је приказан кинематски модел диференцијалног робота.



Слика 3.2 Кинематски модел диференцијалног робота

3.2 Rosbag

ROS (Robot Operating System) је *open-source* екосистем који обезбеђује оквир, алате и библиотеке за развој, постављање, извршавање и одржавање роботских апликација. Чворови у оквиру *ROS 2* представљају основне јединице извршавања и сваки чвор треба да извршава један логички задатак. Топици представљају један од три типа интерфејса између чворова и користе се за континуалне токове података, као што су подаци са сензора [23].

На роботу је имплементиран *ROS 2* који је коришћен за комуникацију између различитих софтверских и хардверских компоненти. Погонски и одометријски системи су реализовани на микроконтролеру који комуницира са рачунаром на којем је *ROS*, посредством чвора задуженог за комуникацију. Та комуникација се одвија фреквенцијом од 10 Hz и микроконтролер шаље податке о одометрији на рачунар. С обзиром на то да се одометрија рачуна на већој фреквенцији унутар микроконтролера, на *ROS* се шаље прорачуната позиција и оријентација, као и просечна вредност линеарне и угаоне брзине за проширени временски интервал комуникације.

Rosbag је алат који служи за снимање података који се објављују на топицима. Прикупљени подаци се чувају у бази података. Поред самих података, чува се и време објављивања. Снимљени подаци се касније могу репродуковати, чиме се репродукује ток података забележен током рада робота у реалном времену. То омогућава да се различити алгоритми тестирају на идентичном скупу података, због чега њихови резултати могу директно да се пореде [24].

Топици који се снимају наводе се на почетку снимања *rosvbag*-а. За прављење скупа података за овај рад, снимани су следећи топици:

- одометрија са пасивних точкова,
- секвенца слика са камере и
- параметри калибрације камере.

3.3 Камера

За снимање секвенци у експерименту, на робота је постављена монокуларна камера *OV9281*. Ова камера је типа *global shutter*, односно сви пиксели у слици се снимају у истом тренутку и током истог времена експозиције. Већина камера, као што су оне у телефонима, су типа *rolling shutter*. Оне секвенцијално снимају редове слике. Експозиција у таквим камерама започиње у каснијем тренутку за сваки наредни ред у слици. То доводи до изобличења слике док се камера креће [25]. Због тога су *global shutter* камере погодније за визуелну одометрију и зато је *OV9281* изабрана за овај рад. *OV9281* камера је монохроматска, односно снима црно-беле слике. Резолуција је 1280×720 пиксела, а снимање је вршено брзином од 120 слика у секунди.

3.3.1 Калибрација камере

Стварна жижна даљина камере представља растојање између центра пројекције и пројективне равни слике. С обзиром на то да појединачни пиксели на сензору не морају бити идеалног квадратног облика, жижна даљина се представља са два параметра f_x и f_y . Жижна даљина f_x представља производ стварне жижне даљине и броја пиксела по дужини, док f_y представља производ стварне жижне даљине и броја пиксела по висини. Основна тачка слике је место у којем нормала из центра пројекције на раван слике продире пројективну раван. Основна тачка не мора да се поклапа са центром слике. То одступање се изражава параметрима c_x и c_y , који представљају координате основне тачке [26]. Унутрашњи параметри камере се могу представити матрицом датом у (14).

$$M = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (14)$$

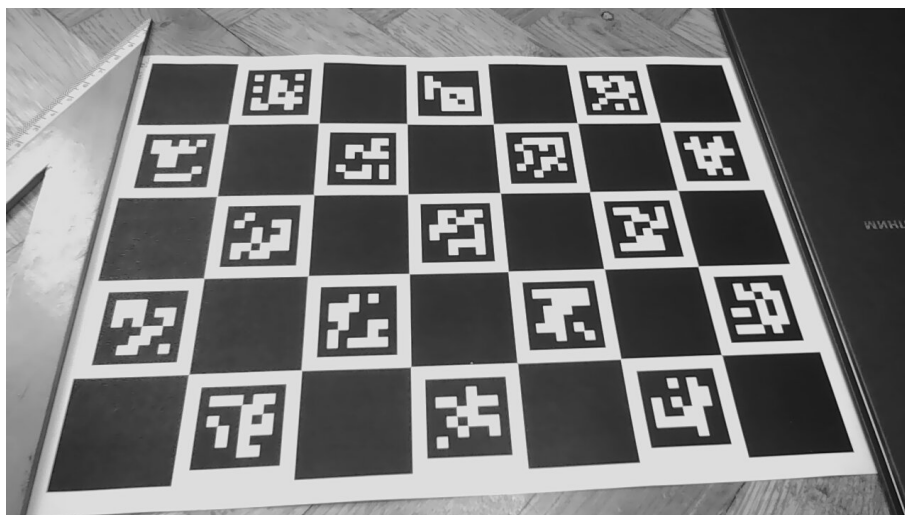
Сочиво у камери служи да се светлост са веће површине скупи и фокусира на сензор. Без сочива би се веома мала количина светлости сакупљала на сензору, што би захтевало веома дуго време експозиције. Идеално сочиво не би уносило никакво изобличење слике. Такво сочиво би било идеалне геометрије и идеално поравнато са сензором. Међутим, у пракси ниједно сочиво није идеално. Сферна сочива су много једноставнија за израду од параболичних и веома је тешко прецизно поравнати сочиво са сензором. Два главна типа изобличења која сочива уводе су радијално и тангенцијално изобличење. Радијално изобличење настаје као резултат неидеалног облика сочива, док тангенцијално изобличење настаје као резултат неидеалне монтаже сочива у односу на сензор [26]. Коефицијенти изобличења могу да се

представе вектором датом у (15). Коефицијенти k_1 , k_2 и k_3 представљају коефицијенте радијалног изобличења, док p_1 и p_2 представљају коефицијенте тангенцијалног изобличења.

$$d = \begin{bmatrix} k_1 & k_2 & p_1 & p_2 & k_3 \end{bmatrix} \quad (15)$$

Калибрација камере служи за идентификацију унутрашњих параметара камере и коефицијената изобличења. За калибрацију је коришћена *ChArUco* табла. Табла комбинује шаховску таблу и *ArUco* маркере. Прво се детектују *ArUco* маркери и њихови положаји. Затим се на основу маркера и познате геометрије табле интерполирају унутрашњи ћошкови на шаховској табли, који представљају тачке од интереса за калибрацију. Целокупан процес калибрације је реализован помоћу *OpenCV* библиотеке.

За генерисање табле коришћен је речник маркера *DICT_6X6_250*. Дужина странице једног шаховског поља износи 40 mm, а табла се састоји од пет поља у хоризонталном и седам у вертикалном правцу. *ArUco* маркери се налазе у белим пољима шаховске табле и дужина њихове странице износи 30 mm. За калибрацију је коришћено 50 фотографија табле. Табла се у фотографијама налази на различитим местима у видном пољу камере и под различитим угловима. Једна слика која је коришћена за калибрацију је приказана на слици 3.3.



Слика 3.3 Слика *ChArUco* табле која је коришћена за калибрацију

Калибрацијом камере су одређени унутрашњи параметри камере који су приказани у (16), као и коефицијенти изобличења који су приказани у (17).

$$M = \begin{bmatrix} 1032.90964 & 0 & 681.51361 \\ 0 & 1032.44152 & 317.85746 \\ 0 & 0 & 1 \end{bmatrix} \quad (16)$$

$$d = \begin{bmatrix} -0.002011 & 0.072985 & -0.004572 & 0.002418 & -0.128552 \end{bmatrix} \quad (17)$$

Време експозиције је веома важан параметар који је потребно подесити за успешан рад визуелне одометрије. Време експозиције представља временски интервал у којем је сензор камере изложен светлости и прикупља светлост како би направио слику. Тамне сцене захтевају дуго време експозиције које омогућава да се светлост дуже време сакупља на сензору [27]. Кретање камере током једног времена експозиције доводи до регистровања разли-

читих положаја сцене у истој слици, што резултује замућењем слике (*motion blur*). Замућење зависи од количине кретања током времена експозиције. За кратко време експозиције, чак и при великим брзинама кретања, замућење ће бити мало, док ће за дуго време замућење бити велико чак и при малим брзинама кретања [28].

За визуелну одометрију је потребна оштра слика чак и при великим брзинама, због чега је битно користити кратко време експозиције. За прикупљање скупа података коришћено је време експозиције од 1,5 ms. Резултујућа слика је била веома тамна, због чега су контраст и гама подешени на вредности 32 и 200, респективно.

3.4 Секвенце

За прикупљање података робот се кретао по столу за такмичење Еуробот 2026 који се налази у лабораторији ГЗ у Машинском институту. Лабораторија представља богато визуелно окружење које је погодно за прикупљање података који ће бити коришћени за анализу визуелне одометрију. На слици 3.4 је приказан изглед лабораторије забележен камером на роботу током једне од снимљених секвенци.



Слика 3.4 Изглед лабораторије забележен камером на роботу током снимања секвенце

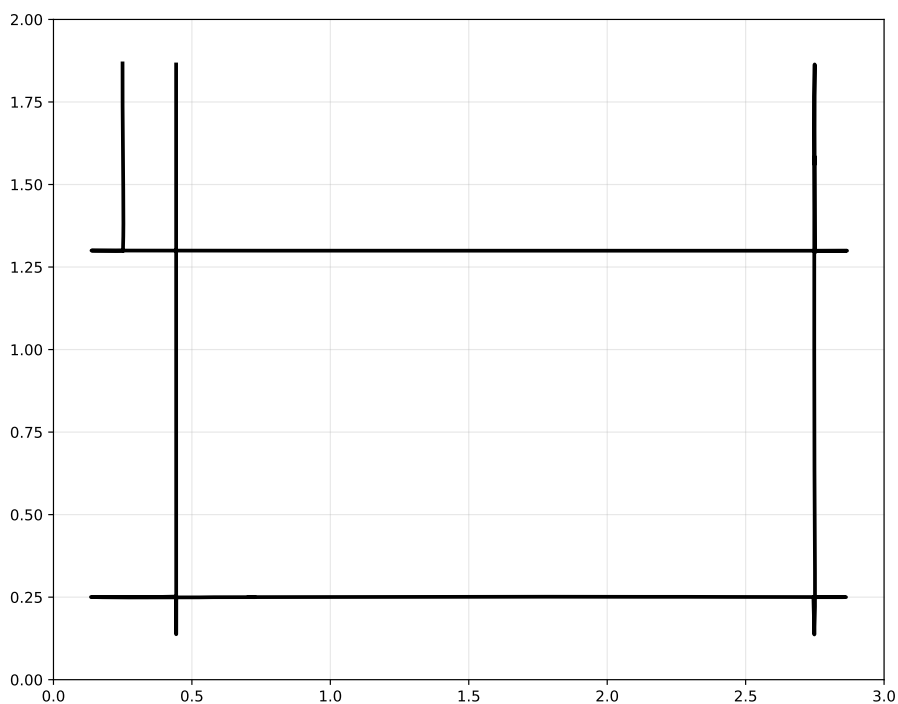
Сто по којем се кретао робот је приказан на слици 3.5. Његове димензије су тачно дефинисане: ширина је 2 m и дужина је 3 m. Уз ивицу стола се простире зид висине 7 cm.



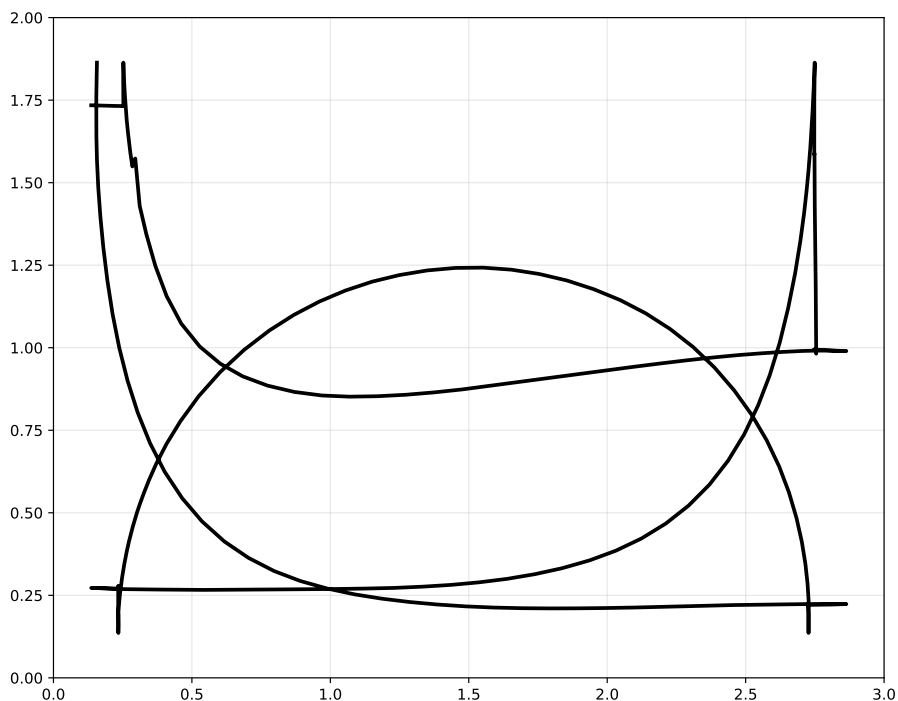
Слика 3.5 Изглед стола за такмичење Еуробот 2026

Снимљена су два типа секвенци. Први тип се састоји искључиво од праволинијских translација и ротације око центра робота. Други тип користи дугачке криволинијске путање и кратке праволинијске translације, као и ротацију око центра. Оба типа обухватају две секвенце у којима робот пролази кроз исте путање, али са различитим максималним брзинама. У споријим секвенцама максимална линеарна брзина износи $0,5 \frac{\text{m}}{\text{s}}$, а максимална угаона брзина приликом ротације око центра робота износи $\frac{\pi}{2} \frac{\text{rad}}{\text{s}}$. Максимална линеарна и угаона брзина у бржим секвенцама износе $1 \frac{\text{m}}{\text{s}}$ и $\pi \frac{\text{rad}}{\text{s}}$. Угаона брзина при криволинијском кретању није експлицитно ограничена максималном угаоном брзином већ се скалира на основу максималне линеарне брзине.

На слици 3.6 је приказана путања којом се робот кретао у праволинијским секвенцама, док је на слици 3.7 приказана путања у криволинијским секвенцама.



Слика 3.6 Референтна путања робота у праволинијским секвенцама

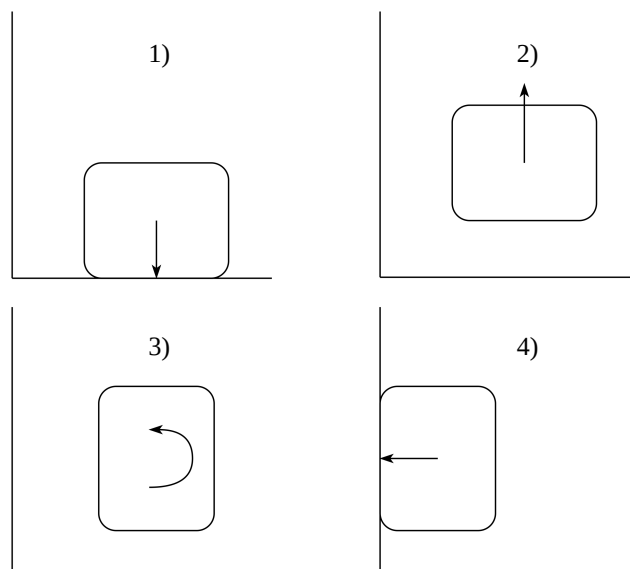


Слика 3.7 Референтна путања робота у криволинијским секвенцама

Оваква подела секвенци је изабрана како би се испитао утицај различитих типова кретњи и максималних брзина на успешност визуелне одометрије. Уколико би се секвенце састојале искључиво од праволинијских транслација или искључиво од ротација око центра, резултати не би могли визуелно да се прикажу. Секвенца која би се састојала искључиво од криволинијских кретања би имала другачији проблем који настаје као резултат грешке одометрије са точкова који је објашњен у поглављу 3.1.2. Наиме, при криволинијској кретњи, чак и са идеално идентификованим параметрима одометрије, прорачуната путања одступа од стварне путање којом се робот креће. Уколико бисмо ту путању без корекције користили као референтну, грешка референтне путање би ушла у евалуацију. Такви резултати не би показивали стварну тачност визуелне одометрије. Због тога је пре евалуације визуелне одометрије потребно извршити процену и кориговање референтних путања, што је објашњено у следећем поглављу.

3.4.1 Процена референтних путања

С обзиром на то да су димензије стола тачно дефинисане и да је на ободу стола зид, у тренутку када робот удари у зид једна његова координата и оријентација су тачно одређене. Кратком секвенцом у којој робот удари у два зида која се налазе под правим углом можемо да добијемо референтну тачку где је потпуно познат положај робота (уз минималну грешку која се нагомила у кратким покретима извлачења). Ова кратка секвенца је коришћена након сваке кретње робота и приказана је на слици 3.8. Референтне тачке су коришћене првобитно да би се проверила грешка одометрије са точкова. Затим, уколико је прорачуната грешка значајна, референтне тачке су коришћене за добијање референтне путање.



Слика 3.8 Секвенца добијања референтних тачака

У обе праволинијске секвенце, укупна нагомилана грешка у позицији износи око 2 mm, док грешка у оријентацији износи око $0,26^\circ$. Због веома мале вредности грешке у праволинијским секвенцама, одометрија са тачкова је директно коришћена као референтна путања. У криволинијској путањи грешке у позицији су значајно веће. Са мањим максималним брзинама, грешка у позицији износи око 4,5 mm, а грешка у оријентацији износи $0,007^\circ$. У криволинијској секвенци са већим максималним брзинама грешка у позицији износи 19,5 mm, а у оријентацији $0,2^\circ$. Иако је чак и у криволинијској путањи грешка поприлично мала, с обзиром на то да је значајно већа него у праволинијској путањи, одрађена је корекција пре њеног коришћења као референтне путање.

За процену референтне путање кретање је посматрано у засебним сегментима. Сегменти се састоје искључиво од криволинијских кретања или од праволинијске translације заједно са ротацијом око центра робота. Крајње тачке сваког сегмента су референтне тачке. Израчуната је трансформација крајње тачке, у односу на почетну, која би је довела у референтну. Позиције свих осталих тачака у сегменту у односу на почетну су одређене коришћењем исте трансформације. Сегменти који се састоје од праволинијске translације и ротације око центра робота су сматрани идеалним, због веома мале грешке на целокупној путањи која се састоји само од њих. Трансформација за те сегменте састојала се само од translације и ротације за вредности претходно нагомилане грешке. Трансформација криволинијских сегмената обухвата и скалирање.

Оваква процена референтне путање одржава континуалност путање и задовољава позицију робота у референтним тачкама. Међутим, за све остале положаје у сегменту није могуће тачно одредити референтну путању. Оваква процена уводи претпоставку да се грешка одометрије у криволинијским кретањима равномерно нагомилава дуж сегмента и да може да се отклони јединственом трансформацијом за цео сегмент. Заправо, количина грешке која се нагомилава у сваком тренутку зависи од тренутних линеарних и угаоних брзина робота. Ипак, с обзиром на то да су добијене грешке веома мале, ова апроксимација се сматра довољно добром за добијање референтне путање која ће се користити за евалуацију визуелне одометрије.

4 Резултати и дискусија

4.1 Евалуација одометрије

За потребе евалуације коришћен је алат *KITTI Odometry Evaluation Toolbox* [29, 30], који је првенствено намењен за евалуацију путања одометрије на *KITTI* скупу података. *KITTI Vision Benchmark Suite* је скуп података развијен на платформи за аутономну возњу, који се користи за евалуацију различитих задатака машинске визије и процене кретања [22]. *KITTI Odometry Evaluation Toolbox* је проширен да прихвати и друге референтне путање, за које ће бити вршена евалуација. Поред референтне путање, као улаз се прослеђује и процењена путања. Путање су записане у *KITTI* формату. Свака путања је представљена као низ поза камере. Једна поза је представљена матрицом $\mathbf{T}$ димензија 3×4 , која је приказана у (18). Матрица $\mathbf{T}$ се састоји од матрице ротације димензије 3×3 у левом делу и вектора транслације у последњој колони. Референтна и процењена путања се прослеђују као текстуалне датотеке.

$$\mathbf{T} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \quad (18)$$

Пре сваке позе у улазним датотекама се додаје индекс i који служи за упаривање одговарајућих поза у процењеној и референтној путањи. За сваку позу, матрица $\mathbf{T}$ је записана у једном реду у облику који је приказан у (19).

$$\mathbf{T}_{\text{inline}} = \left[i \quad r_{11} \quad r_{12} \quad r_{13} \quad t_x \quad r_{21} \quad r_{22} \quad r_{23} \quad t_y \quad r_{31} \quad r_{32} \quad r_{33} \quad t_z \right] \quad (19)$$

За евалуацију визуелне одометрије коришћено је следећих пет критеријума:

- просечна транслациона грешка t_{err} ,
- просечна ротациона грешка r_{err} ,
- апсолутна грешка путање АТЕ,
- релативна грешка позе за транслацију $\text{RPE}_{\text{trans}}$ и
- релативна грешка позе за ротацију RPE_{rot} .

Просечна транслациона грешка t_{err} се рачуна као просечна вредност транслационих грешака за сегменте различитих дужина. За сваки сегмент се рачуна растојање између релативних транслација процењене и референтне путање. То растојање се затим нормализује дужином сегмента. Добијена вредност се изражава у процентима.

Просечна ротациона грешка r_{err} се рачуна као просечна вредност ротационих грешака за сегменте различитих дужина. За сваки сегмент се рачуна угао релативне ротације између процењене и референтне путање. Угао се затим нормализује дужином сегмента. Добијена вредност се изражава у степенима на 100 m. Међутим, због релативно кратких путања у односу на путање из *KITTI* скупа података, у овом раду се добијена вредност просечне ротационе грешке изражава у степенима на 1 m.

Апсолутна грешка путање АТЕ се рачуна на основу разлике између положаја процењене и референтне путање за сваку одговарајућу позу. За сваку позу се рачуна величина

грешке положаја, чији се квадрат затим усредњава. Као резултат се добија корен средње квадратне грешке позиција. Изражава се у метрима.

Релативна грешка позе за translацију RPE_{trans} се рачуна на основу разлике између релативних translација процењене и референтне путање за узастопне позе. Као резултат се добија средња вредност растојања између одговарајућих релативних translација, изражена у метрима.

Релативна грешка позе за ротацију RPE_{rot} се рачуна на основу разлике између релативних ротација процењене и референтне путање за узастопне позе. Као резултат се добија средња вредност углова између одговарајућих релативних ротација. Изражава се у степенима.

4.1.1 Поравнавање процењене и референтне путање

Циљ поравнавања је довођење процењене путање у одговарајући положај и оријентацију у односу на референтну путању пре евалуације. Путање могу бити представљене у различитим координатним системима. Референтне путање, које су настале обрадом одометрије са пасивних тачкова, су представљене у координатном систему стола. Процењене путање, које су настале као резултат алгоритама визуелне одометрије, су представљене у локалном координатном систему камере, који не мора бити оријентисан на исти начин за сваки алгоритам.

С обзиром на то да монокуларна визуелна одометрија може да процени кретање само до фактора скале, за евалуацију резултата је потребно додатно скалирати процењене путање [2]. Скалирање се врши у оквиру *Umeyama* методе, која одређује фактор скале заједно са translацијом и ротацијом тако да се минимизује средња квадратна грешка између одговарајућих тачака две путање [31]. Због неодређености апсолутне скале и накнадног скалирања процењене путање према референтној, оваква евалуација не оцењује способност алгоритма да самостално одреди апсолутну скалу кретања. Евалуација пре свега омогућава процену колико доследно процењена путања прати кретање приказано референтном путањом.

За поравнавање процењене и референтне путање коришћена је *Umeyama* метода. Ова метода је део алата *KITTI Odometry Evaluation Toolbox* и описана је у следећем поглављу.

4.1.2 *Umeyama* метода

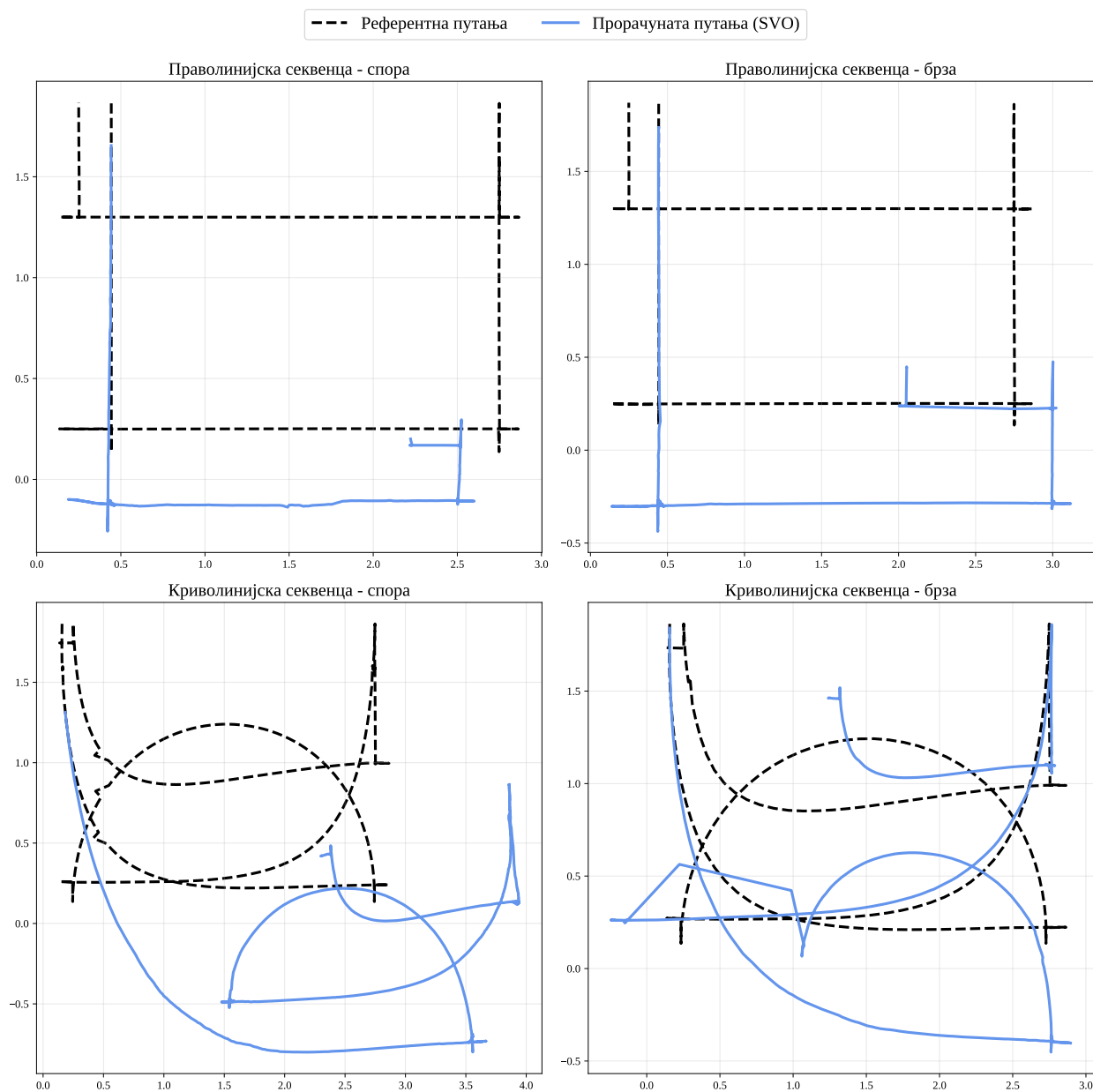
Из процењене и референтне путање се првобитно издвајају положаји камере, при чему се одговарајући положаји из обе путање посматрају као парови тачака. За сваку групу тачака се израчунава средња вредност координата. Добијене средње вредности се користе за центрирање тачака, чиме се положаји посматрају у односу на средиште одговарајуће групе.

Након центрирања формира се матрица коваријације. Матрица коваријације описује међусобну повезаност координата парова центрираних тачака. Над овом матрицом се врши сингуларна декомпозиција (*SVD*), на основу које се одређује матрица ротације која оптимално поравнава две групе тачака [31].

Након одређивања ротације израчунава се фактор скале. Он се одређује на основу расподеле тачака око њихових средњих вредности и израчунате ротације тако да се минимизује средња квадратна грешка између одговарајућих тачака [31]. На крају се израчунава translација на основу средњих вредности две групе тачака, ротације и фактора скале [31].

4.2 Резултати *SVO*

У криволинијској секвенци са већим максималним брзинама је дошло до једног губитка праћења. Губитак праћења се десио при ротацији око центра робота. *SVO* је остварио задовољавајућу процену ротације, како на локалном тако и на глобалном нивоу. На тај резултат нам указују веома мале вредности r_{err} и RPE_{rot} . Веома велике вредности t_{err} и АТЕ нам указују да *SVO* алгоритам није у стању да поуздано одреди апсолутну скалу translације. До таквог закључка се може доћи и на основу приказа процењених путања који су дати на слици 4.1.

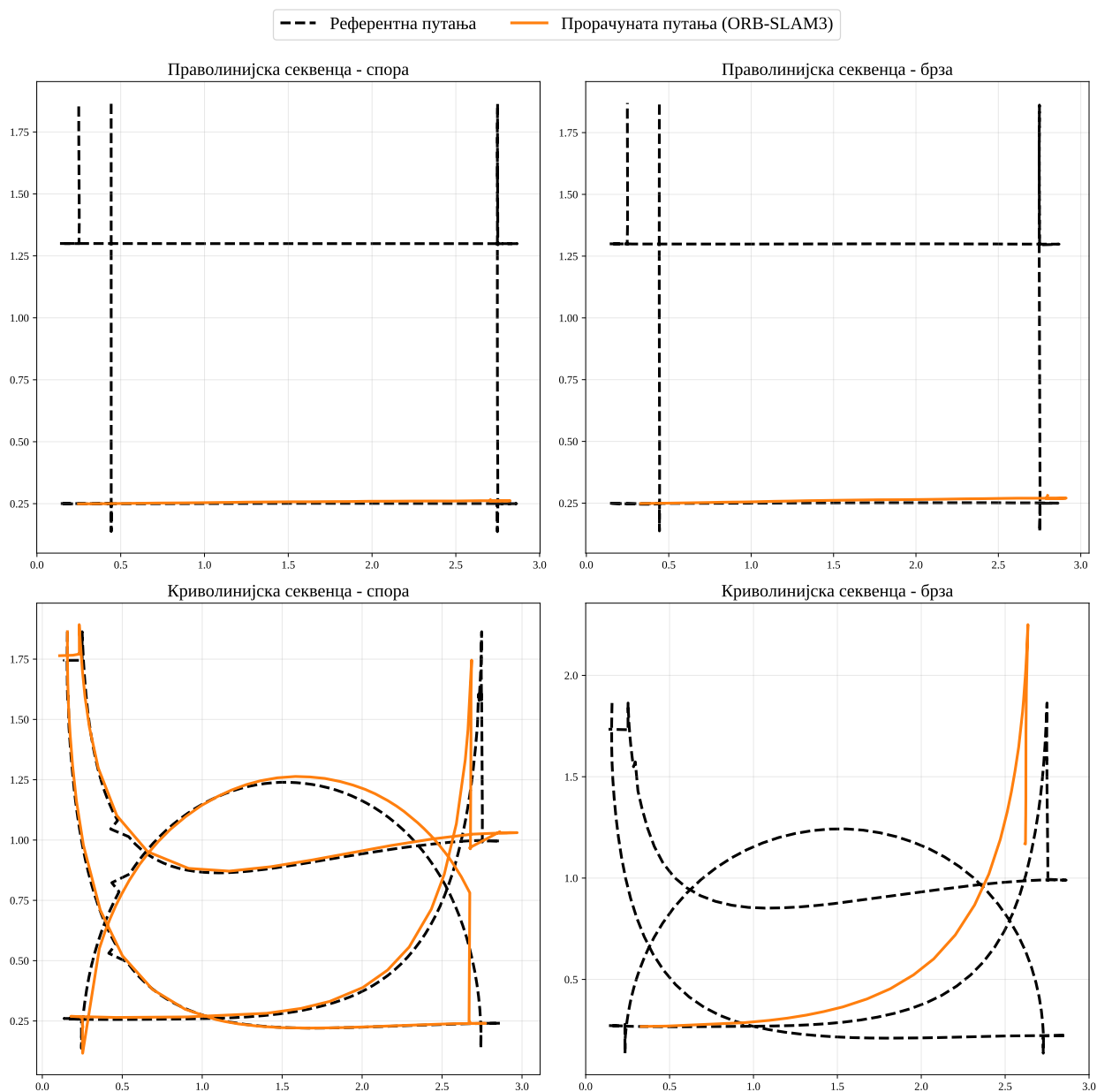


Слика 4.1 Путање процењене алгоритмом *SVO*

Примећује се да сваки сегмент у процењеним путањама одржава приближно исту оријентацију као референтна путања. Међутим, процењена релативна скала се стално мења и долази до значајних одступања процењених од референтних путања.

4.3 Резултати *ORB-SLAM3*

У *ORB-SLAM3* алгоритму је долазило до губитка праћења на крају сваке ротације око центра робота. У спорој криволинијској секвенци, током успешног праћења, долазило је до поновног уочавања већ виђених делова сцене, због чега је *ORB-SLAM3* успешно извршио локализацију. Након тога није долазило до губитка праћења. Одлично подудараре процене и референтне путање се може уочити на слици 4.2. Сви критеријуми сем RPE_{rot} за ту секвенцу имају веома мале грешке. На основу тога можемо да закључимо да *ORB-SLAM3* показује веома добру глобалну процену кретања. Међутим нешто већа вредност RPE_{rot} указује на лошију локалну процену кретања.



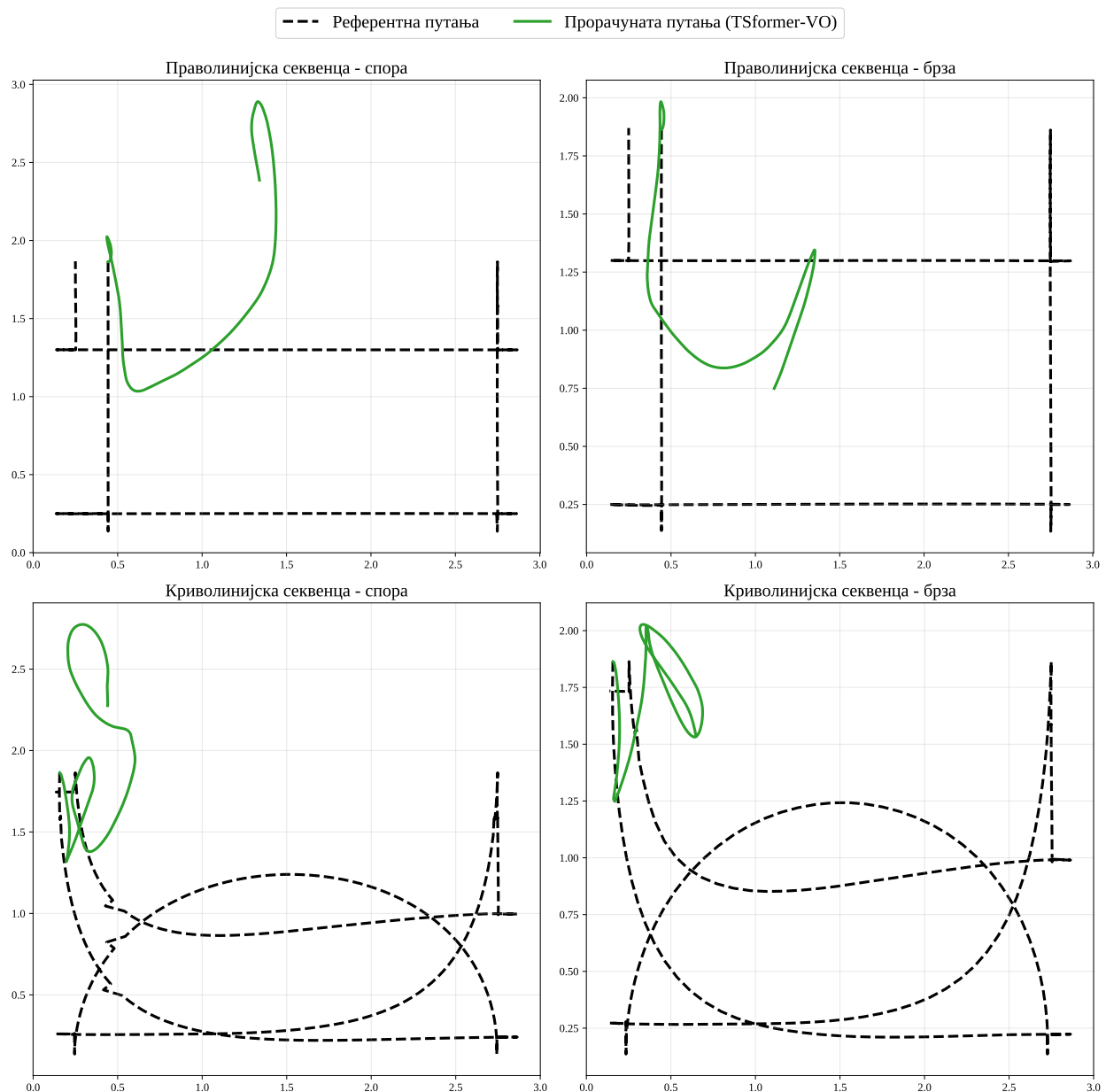
Слика 4.2 Путање процене алгоритмом *ORB-SLAM3*

У осталим секвенцама су приказани само најдужи сегменти, због губитка праћења и неизвршене локализације. Због тога вредности критеријума нису у потпуности репрезентативне за успешност рада алгоритма. Упркос томе, вредности свих критеријума при бржим

секвенцама су значајно веће. На основу тога можемо да закључимо да је *ORB-SLAM3* алгоритам осетљив на брзину кретања: са порастом брзине смањује се успешност процене кретања.

4.4 Резултати *TSformer-VO*

Ни у једној од секвенци алгоритам *TSformer-VO* није успео да смислено процени кретање. На слици 4.3 су приказане референтне путање и путање које је проценио алгоритам *TSformer-VO*.



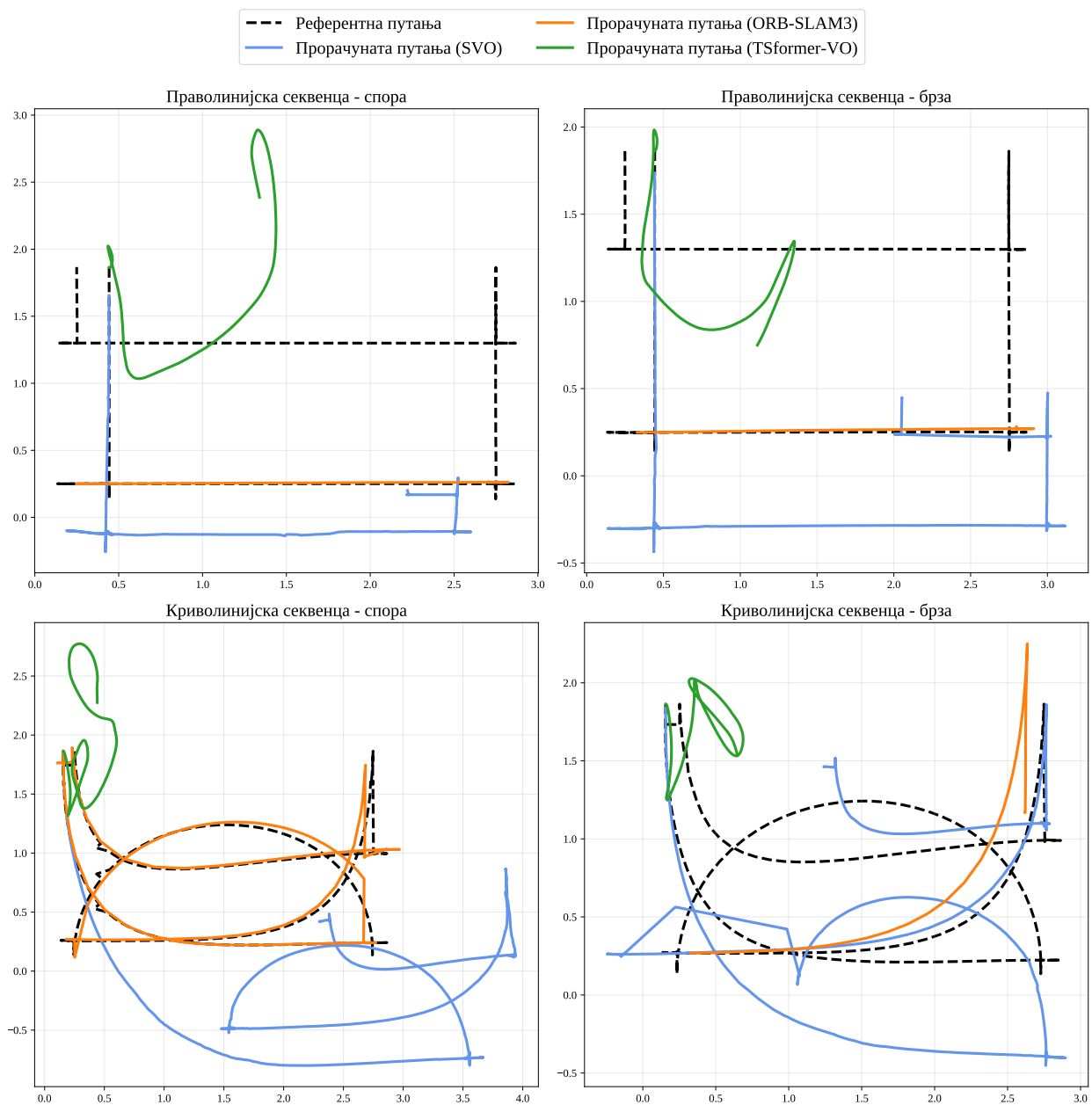
Слика 4.3 Путање процењене алгоритмом *TSformer-VO*

Процењене путање не показују сличности са референтним путањама. Један од могућих разлога за овакве резултате је то што је коришћени модел обучен на скупу података који се значајно разликује од података коришћених у овом експерименту. Модел је обучаван на *KITTI* скупу података, који садржи снимке прикупљене током вожње возила у саобраћају [18, 22], док су у овом раду коришћени снимци мобилног робота у лабораторијском окружењу.

Будући да нисам био у могућности да обучим модел на сопственом скупу података, можемо закључити само да је *TSformer-VO* веома осетљив на скуп података на којем је обучаван.

4.5 Поређење алгоритама

На слици 4.4 су приказане референтне путање и процењене путање за све алгоритме из овог рада. У табели 4.1 су приказане вредности критеријума који су коришћени за евалуацију.



Слика 4.4 Процењене путање свих анализираних алгоритама

При поређењу алгоритама није разматран критеријум RPE_{trans} , јер је у овом експерименту слабо информативан за поређење, како због малог померања између кадрова, тако и због тога што губици праћења утичу на репрезентативност његове вредности.

На основу резултата се може уочити да *ORB-SLAM3* остварује најбоље подударање са референтним путањама, али пре успешне локализације долази до константних губитака

праћења у ротацијама и алгоритам је осетљив на брзину кретања робота. *SVO* алгоритам показује најбољу процену оријентације, али није у стању да одреди скалу translације. *TSformer-VO* показује највеће одступање од референтних путања, без обзира на тип секвенце и без обзира на брзину кретања.

Табела 4.1 Резултати евалуације визуелне одометрије

Метода	Секвенца				
		Праволинијска		Криволинијска	
		Спора	Брза	Спора	Брза
t_{err} [%]	SVO	45.692	22.981	12.084	2.529*
	ORB-SLAM3	1.136*	2.643*	3.547**	8.063*
	TSformer-VO	56.442	60.369	69.122	70.707
r_{err} [°/m]	SVO	0.053	0.053	0.350	0.006*
	ORB-SLAM3	6.875*	13.604*	0.046**	2.257*
	TSformer-VO	11.528	10.762	11.178	8.351
ATE [m]	SVO	1.199	1.234	1.576	0.689*
	ORB-SLAM3	0.056*	0.145*	0.066**	0.494*
	TSformer-VO	2.202	2.182	2.187	2.362
RPE _{trans} [m]	SVO	0.005	0.005	0.004	0.008*
	ORB-SLAM3	0.009*	0.017*	0.016**	0.060*
	TSformer-VO	0.003	0.003	0.003	0.003
RPE _{rot} [°]	SVO	0.222	0.360	0.169	0.795*
	ORB-SLAM3	0.857*	1.569*	0.870**	2.896*
	TSformer-VO	0.222	0.225	0.319	0.336

* За означене резултате, алгоритам је током секвенце изгубио праћење.

** За означене резултате, алгоритам је током секвенце изгубио праћење, али је касније успешно извршио локализацију.

5 Закључак

У овом раду је испитана примена монокуларне визуелне одометрије као решење за процену кретања мобилног робота у затвореном простору. Анализирани су представници различитих типова метода који користе само секвенцу слика са камере за процену путање. Ради адекватне анализе прикупљен је експериментални скуп података на већ постојећем мобилном роботу. Прикупљени скуп података обухвата референтне путање по којима се робот кретао и секвенце слика са камере које су прикупљене током тог кретања. Кретање робота је изабрано на такав начин да би се испитао утицај различитих типова кретања и различитих максималних брзина. Процењене путање су поређене са референтним коришћењем постојећих алата и критеријума.

SVO алгоритам је показао добру процену оријентације. Међутим, није у стању да поуздано процени скалу трансформације, што доводи до велике грешке положаја. *ORB-SLAM3* је услед успешне локализације кроз *vSLAM* показао добро подударање процењене путање са референтном. Међутим, долазило је до губитка праћења током ротација, а алгоритам је показао осетљивост на брзину кретања. *TSformer-VO* није у стању да у задатим условима процени кретање. Вероватан разлог за то је велика разлика између скупа података на којима је обучаван и скупа података на којима је анализиран.

Монокуларна визуелна одометрија је приказана као потенцијално решење проблема процене кретања. Камере представљају релативно јефтине сензоре који обезбеђују велику количину података великом фреквенцијом. Поред процене кретања, прикупљене слике могу да се користе за мапирање и локализацију, као и за решавање других задатака машинске визије. Међутим, монокуларна визуелна одометрија има значајна ограничења која утичу на њену примену у мобилној роботизици. Процена кретања је осетљива на визуелно окружење. Брзина кретања може да утиче на успешност праћења, кроз стварање великог помераја између узастопних слика и због замућења слике које може да изазове. У случају губитка праћења, систем који процењује положај искључиво визуелном одометријом остаје потпуно без информација о кретању до следеће иницијализације. Неодређеност апсолутне скале омогућава само релативну процену трансформације. Због тога, монокуларна визуелна одометрија је погоднија као део система за локализацију него као самосталан извор процене положаја.

Добијени резултати остављају простор за даље испитивање монокуларне визуелне одометрије. Евалуација би могла да се прошири на дуже секвенце кретања и на већ мапиране просторе, како би се детаљније анализирао утицај локализације на процену кретања. Различите оријентације камере нису испитане због саме конструкције робота. Камера која је оријентисана ка подлози која се налази на константном растојању могла би да омогући додатну метричку информацију о кретању и тиме смањи утицај неодређености апсолутне скале. Није испитан ни утицај фузије са другим сензорима. Визуелна одометрија се често комбинује са инерцијалним сензорима, који могу да обезбеде додатне информације о кретању и допринесу поузданијој процени.

Литература

- [1] David Nistér, Oleg Naroditsky и James Bergen. "Visual Odometry". У: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (2004.), стр. 652–659.
- [2] Davide Scaramuzza и Friedrich Fraundorfer. "Visual Odometry: Part I - The First 30 Years and Fundamentals". У: *IEEE Robotics & Automation Magazine* 18.4 (дец. 2011.), стр. 80–92. DOI: [10.1109/MRA.2011.943233](https://doi.org/10.1109/MRA.2011.943233).
- [3] Ioannis Kostavelis и др. "Stereo-Based Visual Odometry for Autonomous Robot Navigation". У: *International Journal of Advanced Robotic Systems* 13.1 (2016.), стр. 1–19. DOI: [10.5772/62099](https://doi.org/10.5772/62099).
- [4] Kourosh Khoshelham и Sander Oude Elberink. "Accuracy and Resolution of Kinect Depth Data for Indoor Mapping Applications". У: *Sensors* 12.2 (2012.), стр. 1437–1454. DOI: [10.3390/s120201437](https://doi.org/10.3390/s120201437).
- [5] Chao Duan и др. "Deep Learning for Visual SLAM in Transportation Robotics: A Review". У: *Transportation Safety and Environment* 1.3 (2019.), стр. 177–184. DOI: [10.1093/tse/tdz019](https://doi.org/10.1093/tse/tdz019).
- [6] Christian Forster, Matia Pizzoli и Davide Scaramuzza. "SVO: Fast Semi-Direct Monocular Visual Odometry". У: *2014 IEEE International Conference on Robotics and Automation (ICRA)*. 2014., стр. 15–22. DOI: [10.1109/ICRA.2014.6906584](https://doi.org/10.1109/ICRA.2014.6906584).
- [7] Zaixing He и др. "Spatiotemporal visual odometry using ground plane in dynamic indoor environment". У: *Optik* 220 (2020.), стр. 165165. DOI: [10.1016/j.ijleo.2020.165165](https://doi.org/10.1016/j.ijleo.2020.165165).
- [8] Myriam Servières и др. "Visual and Visual-Inertial SLAM: State of the Art, Classification, and Experimental Benchmarking". У: *Journal of Sensors* 2021 (2021.), стр. 2054828. DOI: [10.1155/2021/2054828](https://doi.org/10.1155/2021/2054828).
- [9] Christian Forster и др. "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems". У: *IEEE Transactions on Robotics* 33.2 (2017.), стр. 249–265. DOI: [10.1109/TRO.2016.2623335](https://doi.org/10.1109/TRO.2016.2623335).
- [10] Hailin Jin, Paolo Favaro и Stefano Soatto. "A Semi-Direct Approach to Structure from Motion". У: *The Visual Computer* 19.6 (2003.), стр. 377–394. DOI: [10.1007/s00371-003-0202-6](https://doi.org/10.1007/s00371-003-0202-6).
- [11] Nicholas D. Molton, Andrew J. Davison и Ian Reid. "Locally Planar Patch Features for Real-Time Structure from Motion". У: *Proceedings of the British Machine Vision Conference*. 2004. DOI: [10.5244/C.18.90](https://doi.org/10.5244/C.18.90).
- [12] Geraldo Silveira, Ezio Malis и Patrick Rives. "An Efficient Direct Approach to Visual SLAM". У: *IEEE Transactions on Robotics* 24.5 (2008.), стр. 969–979. DOI: [10.1109/TRO.2008.2004829](https://doi.org/10.1109/TRO.2008.2004829).
- [13] Christopher Mei и др. "Efficient Homography-Based Tracking and 3-D Reconstruction for Single Viewpoint Sensors". У: *IEEE Transactions on Robotics* 24.6 (2008.), стр. 1352–1364. DOI: [10.1109/TRO.2008.2007941](https://doi.org/10.1109/TRO.2008.2007941).

- [14] Robotics and Perception Group, University of Zurich. *SVO Pro: Semi-direct Visual-Inertial Odometry and SLAM for Monocular, Stereo, and Wide Angle Cameras*. 2021. URL: https://github.com/uzh-rpg/rpg_svo_pro_open (посећено 9. 2026.).
- [15] Carlos Campos и др. "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM". У: *IEEE Transactions on Robotics* 37.6 (2021.), стр. 1874–1890. DOI: [10.1109/TRO.2021.3075644](https://doi.org/10.1109/TRO.2021.3075644).
- [16] Raúl Mur-Artal, J. M. M. Montiel и Juan D. Tardós. "ORB-SLAM: A Versatile and Accurate Monocular SLAM System". У: *IEEE Transactions on Robotics* 31.5 (2015.), стр. 1147–1163. DOI: [10.1109/TRO.2015.2463671](https://doi.org/10.1109/TRO.2015.2463671).
- [17] Raúl Mur-Artal и Juan D. Tardós. "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras". У: *IEEE Transactions on Robotics* 33.5 (2017.), стр. 1255–1262. DOI: [10.1109/TRO.2017.2705103](https://doi.org/10.1109/TRO.2017.2705103).
- [18] André O. Françani и Marcos R. O. A. Maximo. "Transformer-Based Model for Monocular Visual Odometry: A Video Understanding Approach". У: *IEEE Access* 13 (2025.), стр. 13959–13971. DOI: [10.1109/ACCESS.2025.3531667](https://doi.org/10.1109/ACCESS.2025.3531667).
- [19] Alexey Dosovitskiy и др. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". У: *International Conference on Learning Representations (ICLR)*. 2021.
- [20] Jacob Devlin и др. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". У: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2019., стр. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- [21] Gedas Bertasius, Heng Wang и Lorenzo Torresani. "Is Space-Time Attention All You Need for Video Understanding?" У: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Св. 139. 2021., стр. 813–824.
- [22] Andreas Geiger и др. "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite". У: *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 2012., стр. 3354–3361. DOI: [10.1109/CVPR.2012.6248074](https://doi.org/10.1109/CVPR.2012.6248074).
- [23] Open Robotics. *ROS 2 Humble Documentation*. 2026. URL: <https://docs.ros.org/en/humble/index.html> (посећено 9. 2026.).
- [24] Open Robotics. *Recording and Playing Back Data*. 2026. URL: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html> (посећено 9. 2026.).
- [25] David Schubert и др. "Direct Sparse Odometry with Rolling Shutter". У: *Proc. of the European Conf. on Computer Vision (ECCV)*. 2018., стр. 699–714. ISBN: 9783030012373. DOI: [10.1007/978-3-030-01237-3_42](https://doi.org/10.1007/978-3-030-01237-3_42). eprint: <https://arxiv.org/abs/1808.00558>.
- [26] Милутин Николић. *Модел камере и калибрација*. Наставни материјал из предмета Напредна роботика, мастер студије, летњи семестар. 2021.
- [27] J. P. A. M. van der Horst и др. "The Way to Modern Shutter Speed Measurement Methods: A Historical Overview". У: *Sensors* 22.5 (2022.), стр. 1871. DOI: [10.3390/s22051871](https://doi.org/10.3390/s22051871).

- [28] Peidong Liu и др. "MBA-VO: Motion Blur Aware Visual Odometry". У: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021., стр. 5550–5559. DOI: [10.1109/ICCV48922.2021.00550](https://doi.org/10.1109/ICCV48922.2021.00550).
- [29] Huangying Zhan. *KITTI Odometry Evaluation Toolbox*. <https://github.com/Huangying-Zhan/kitti-odom-eval>. GitHub repository. 2019.
- [30] Huangying Zhan и др. "Visual Odometry Revisited: What Should Be Learnt?" У: *arXiv preprint arXiv:1909.09803* (2019.).
- [31] Shinji Umeyama. "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns". У: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4 (апр. 1991.), стр. 376–380. DOI: [10.1109/34.88573](https://doi.org/10.1109/34.88573).

Прилог

Генерисање *ChArUco* табле

```
import numpy as np
import cv2, PIL, os
from cv2 import aruco
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd

dict = cv2.aruco.DICT_6X6_250
v = 7 # Number of squares vertically
h = 5 # Number of squares horizontally
a_mm = 40 # Square side length (in mm)
m_mm = 30 # ArUco marker side length (in mm)
mv_mm = 8.5 # Margins size, vertical (in mm)
mh_mm = 5.0 # Margins size, horizontal (in mm)
print_margin_mm = 5.0 # printer limitation (in mm)

pixel_per_mm = 600.0 / 25.4
a = round(a_mm * pixel_per_mm)
m = round(m_mm * pixel_per_mm)
mv = round((mv_mm - print_margin_mm) * pixel_per_mm)
mh = round((mh_mm - print_margin_mm) * pixel_per_mm)

IMG_SIZE = (h * a, v * a)
OUTPUT_NAME = "ChArUco_A4_7x5_40A_30M.png"

def generate_board():
    dictionary = cv2.aruco.getPredefinedDictionary(dict)
    board = cv2.aruco.CharucoBoard(
        (h, v),
        a,
        m,
        dictionary,
    )
    img = cv2.aruco.CharucoBoard.generateImage(board, IMG_SIZE)
    image = cv2.copyMakeBorder(
        img, mv, mv, mh, mh, cv2.BORDER_CONSTANT, value=(255, 255, 255)
    )
    height, width = image.shape
    confirmed_image_size = (height, width)
    print(str(IMG_SIZE) + " resized to " + str(confirmed_image_size))
    # additional margin for printing, empirical
    mh_print = 50
    mv_print = round(mh_print * 297.0 / 210.0)
```

```

print_image = cv2.copyMakeBorder(
    image, mv, mv, mh, mh, cv2.BORDER_CONSTANT, value=(255, 255, 255)
)
cv2.imwrite(OUTPUT_NAME, print_image)

```

```
generate_board()
```

Израчунавање параметара камере

```

import numpy as np
import cv2, PIL, os
from cv2 import aruco
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
import json

dict = cv2.aruco.DICT_6X6_250
v = 7
h = 5
a_mm = 40
m_mm = 30

pixel_per_mm = 600.0 / 25.4
a = round(a_mm * pixel_per_mm)
m = round(m_mm * pixel_per_mm)

directory = "calibration-photos"

def get_calibration_parameters(img_dir):
    dictionary = cv2.aruco.getPredefinedDictionary(dict)
    board = cv2.aruco.CharucoBoard((h, v), a, m, dictionary)
    params = cv2.aruco.DetectorParameters()
    detector = cv2.aruco.ArucoDetector(dictionary, params)

    image_files = [
        os.path.join(img_dir, f) for f in os.listdir(img_dir) if f.endswith(".jpg")
    ]
    all_charuco_ids = []
    all_charuco_corners = []

    for image_file in image_files:
        image = cv2.imread(image_file)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        imgSize = image.shape
        image_copy = image.copy()
        marker_corners, marker_ids, rejectedCandidates = detector.detectMarkers(image)

        if len(marker_ids) > 0:
            print("len(marker_ids)" + str(len(marker_ids)))
            ret, charucoCorners, charucoIds = cv2.aruco.interpolateCornersCharuco(
                marker_corners, marker_ids, image, board
            )

            if charucoIds is not None and len(charucoCorners) > 3:
                all_charuco_corners.append(charucoCorners)
                all_charuco_ids.append(charucoIds)

    result, mtx, dist, rvecs, tvecs = cv2.aruco.calibrateCameraCharuco(
        all_charuco_corners, all_charuco_ids, board, imgSize, None, None
    )
    return mtx, dist

OUTPUT_JSON = "calibration.json"

mtx, dist = get_calibration_parameters(directory)

```

```
data = {"mtx": mtx.tolist(), "dist": dist.tolist()}

with open(OUTPUT_JSON, "w") as json_file:
    json.dump(data, json_file, indent=4)
```

Издавање одометрије из свих *rosvag* датотека

```
from bagpy import bagreader
import pandas as pd
import math
import os
from pathlib import Path

def get_yaw_from_quaternion(x, y, z, w):
    siny_cosp = 2.0 * (w * z + x * y)
    cosy_cosp = 1.0 - 2.0 * (y * y + z * z)
    return math.atan2(siny_cosp, cosy_cosp)

rootdir = "../bags"
extensions = ".bag"
for subdir, dirs, files in os.walk(rootdir):
    for file in files:
        ext = os.path.splitext(file)[-1].lower()
        if ext in extensions:
            print(os.path.join(subdir, file))
            b = bagreader(f'../bags/{file}')

            csvfiles = []
            odom_msgs = b.message_by_topic("/ros381/odom")
            csvfiles.append(odom_msgs)

            data = pd.read_csv(csvfiles[0])
            print(data)

            result = pd.DataFrame(
                {
                    "timestamp": data["Time"],
                    "x": data["pose.pose.position.x"],
                    "y": data["pose.pose.position.y"],
                    "phi": data.apply(
                        lambda row: get_yaw_from_quaternion(
                            row["pose.pose.orientation.x"],
                            row["pose.pose.orientation.y"],
                            row["pose.pose.orientation.z"],
                            row["pose.pose.orientation.w"],
                        ),
                        axis=1,
                    ),
                    "v": data["twist.twist.linear.x"],
                    "w": data["twist.twist.angular.z"],
                }
            )

            print(result)
            bag = Path(file).stem
            result.to_csv(f'{bag}.csv', index=False)
            print(f'Exported {len(result)} messages to {bag}.csv')
```

Конверзија трајекторије у *KITTI* формат

```
import os
import sys
import numpy as np
from scipy.spatial.transform import Rotation

def convert(input_file, method):
    output_file = os.path.splitext(input_file)[0] + "_kitti.txt"
```

```

transform = np.array([[0, 0, 1], [-1, 0, 0], [0, -1, 0]])

with open(input_file) as source, open(output_file, "w") as target:
    frame_id = 0
    for line in source:
        if not line.strip() or line.startswith("#"):
            continue

        parts = line.split()
        if method == "orb":
            if len(parts) < 8:
                continue
            position = np.array([float(value) for value in parts[1:4]])
            quaternion = [float(value) for value in parts[4:8]]
            rotation = Rotation.from_quat(quaternion).as_matrix()
            rotation = transform @ rotation @ transform.T
            position = transform @ position
        else:
            if len(parts) < 9:
                continue
            frame_id = int(parts[0])
            position = np.array([float(value) for value in parts[2:5]])
            quaternion = [float(value) for value in parts[5:9]]
            rotation = Rotation.from_quat(quaternion).as_matrix()

        pose = np.eye(4)
        pose[:3, :3] = rotation
        pose[:3, 3] = position
        values = " ".join(f"{value:.12f}" for value in pose[:3].reshape(-1))
        target.write(f"{frame_id} {values}\n")
        frame_id += 1

if len(sys.argv) != 3 or sys.argv[1] not in {"orb", "svo"}:
    sys.exit(1)

convert(sys.argv[2], sys.argv[1])

```

Конверзија референтне путање у *KITTI* формат

```

import math
import os
import sys
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d

def parse_timestamp(value):
    if method == "orb":
        integer = int(value.split(".")[0])
        if integer > 1e12:
            return integer / 1e9
    return float(value)

def convert(gt_file, trajectory_file, method):
    output_file = os.path.splitext(gt_file)[0] + "_kitti.txt"
    ground_truth = pd.read_csv(gt_file).sort_values("timestamp")
    timestamps = []
    frame_ids = []

    with open(trajectory_file) as source:
        for line in source:
            if not line.strip() or line.startswith("#"):
                continue
            parts = line.split()
            if method == "orb":
                timestamps.append(parse_timestamp(parts[0]))
                frame_ids.append(len(frame_ids))

```

```

else:
    if len(parts) < 9:
        continue
    frame_ids.append(int(parts[0]))
    timestamps.append(float(parts[1]))

time = ground_truth["timestamp"].to_numpy()
x = interp1d(time, ground_truth["x"].to_numpy(), fill_value="extrapolate")
y = interp1d(time, ground_truth["y"].to_numpy(), fill_value="extrapolate")
phi = interp1d(
    time,
    np.unwrap(ground_truth["phi"].to_numpy()),
    fill_value="extrapolate",
)

with open(output_file, "w") as target:
    for frame_id, timestamp in zip(frame_ids, timestamps):
        yaw = float(phi(timestamp))
        pose = np.array(
            [
                [math.cos(yaw), -math.sin(yaw), 0, float(x(timestamp))],
                [math.sin(yaw), math.cos(yaw), 0, float(y(timestamp))],
                [0, 0, 1, 0],
            ]
        )
        values = " ".join(f" {value:.12f}" for value in pose.reshape(-1))
        target.write(f"{frame_id} {values}\n")

if len(sys.argv) != 4 or sys.argv[1] not in {"orb", "svo"}:
    sys.exit(1)

method = sys.argv[1]
convert(sys.argv[2], sys.argv[3], method)

```

***Containerfile* за *Ubuntu 20.04* окружење**

```

FROM docker.io/ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN useradd -m -s /bin/bash ubuntu
RUN apt-get update && apt-get upgrade -y && \
    apt-get install -y \
    curl \
    wget \
    git \
    vim \
    nano \
    htop \
    net-tools \
    build-essential \
    software-properties-common \
    apt-utils \
    sudo \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
RUN echo "ubuntu ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers.d/ubuntu && \
    chmod 0440 /etc/sudoers.d/ubuntu
RUN mkdir -p /workspace && \
    chown -R ubuntu:ubuntu /workspace
WORKDIR /workspace
USER ubuntu
RUN sudo apt-get update && \
    sudo apt-get install -y locales && \
    sudo locale-gen en_US.UTF-8 && \
    sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
RUN sudo apt-get update && \
    sudo apt-get install -y \
    neovim \
    ranger \
    && sudo apt-get clean

```

```

RUN sudo apt-get update && \
  sudo apt-get install -y \
  x11-apps \
  xauth \
  libx11-6 \
  libxext6 \
  libxrender1 \
  libxtst6 \
  libxi6 \
  libxkbcommon-x11-0 \
  libgl1-mesa-dri \
  libglx-mesa0 \
  mesa-utils \
  dbus-x11 \
  && sudo apt-get clean
RUN sudo apt-get update && \
  sudo apt-get install -y \
  libgtk-3-0 \
  libgtk-3-bin \
  libcantora-gtk3-module \
  && sudo apt-get clean
RUN sudo apt-get update && \
  sudo apt-get install -y \
  libqt5core5a \
  libqt5gui5 \
  libqt5widgets5 \
  qt5-gtk-platformtheme \
  && sudo apt-get clean
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-tk \
  python3-pyqt5 \
  && sudo apt-get clean
RUN sudo apt-get install -y \
  libboost-all-dev \
  libeigen3-dev
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-opencv \
  libopencv-dev \
  && sudo apt-get clean
RUN sudo DEBIAN_FRONTEND=noninteractive DEBCONF_NONINTERACTIVE_SEEN=true \
  apt-get install -y -o Dpkg::Pre-Install-Pkgs::=/bin/true \
  keyboard-configuration && \
  sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com --recv-keys F42ED6FBAB17C654 || true && \
  sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu focal main" > /etc/apt/sources.list.d/ros-latest.list' && \
  sudo apt-get update && \
  sudo DEBIAN_FRONTEND=noninteractive apt-get install -y --allow-unauthenticated ros-noetic-desktop-full && \
  sudo apt-get clean
RUN sudo apt-get update && \
  sudo apt-get install -y \
  python3-catkin-tools \
  python3-vcstool \
  python3-osrf-pycommon \
  libglew-dev \
  libopencv-dev \
  libyaml-cpp-dev \
  libblas-dev \
  liblapack-dev \
  libsuitesparse-dev \
  && sudo apt-get clean
RUN echo "source /opt/ros/noetic/setup.bash" >>> ~/.bashrc

```